Heart Disease Prediction Using Machine Learning

Milestone 3 – Initial Results, Code Structure, and Model Evaluation

Student:- Kaishiv Joshi

Course:- CIND 830 – Big Data Analytics

Dataset:- Framingham Heart Study Dataset

Objective:- The objective of this notebook is to develop an end-to-end machine learning pipeline for predicting the 10-year risk of coronary heart disease using demographic and clinical variables. This notebook demonstrates data preprocessing, exploratory data analysis, feature engineering, model development, evaluation, and interpretation.

In [1]:
```
# ==============================
# Import Required Libraries
# ==============================

import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from scipy import stats

from sklearn.model_selection import train_test_split

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler

from sklearn.feature_selection import RFE

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
```

```
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report,
    roc_curve
)

from imblearn.over_sampling import SMOTE

import shap

print("Libraries imported successfully.")
```

```
Libraries imported successfully.
```

Why these libraries?

- Pandas and NumPy are used for data manipulation.
- Matplotlib and Seaborn are used for visualization.
- SciPy is used for statistical analysis.
- Scikit-learn provides preprocessing, feature selection, machine learning models, and evaluation metrics.
- SMOTE is used to address class imbalance.
- SHAP is used to explain feature importance in the final model.

Load Dataset

In [5]:
```
df = pd.read_csv(r"C:\Users\kaish\OneDrive\Desktop\My Tmu\CIND820 Big Data Analytic

print("Dataset Loaded Successfully")
df.head()
```

```
Dataset Loaded Successfully
```

Out[5]:

	male	age	education	currentSmoker	cigsPerDay	BPMeds	prevalentStroke	prevalentH
0	1	39	4.0	0	0.0	0.0	0	
1	0	46	2.0	0	0.0	0.0	0	
2	1	48	1.0	1	20.0	0.0	0	
3	0	61	3.0	1	30.0	0.0	0	
4	0	46	3.0	1	23.0	0.0	0	

In [7]:
```
# ==============================
# Dataset Overview
# ==============================

print("Shape of Dataset")
print(df.shape)

print("\n")
```

```
print("Column Names")
print(df.columns)

print("\n")

print("Data Types")
print(df.dtypes)
```

```
Shape of Dataset
(4240, 16)


Column Names
Index(['male', 'age', 'education', 'currentSmoker', 'cigsPerDay', 'BPMeds',
       'prevalentStroke', 'prevalentHyp', 'diabetes', 'totChol', 'sysBP',
       'diaBP', 'BMI', 'heartRate', 'glucose', 'TenYearCHD'],
      dtype='object')


Data Types
male                int64
age                 int64
education         float64
currentSmoker       int64
cigsPerDay        float64
BPMeds            float64
prevalentStroke     int64
prevalentHyp        int64
diabetes            int64
totChol           float64
sysBP             float64
diaBP             float64
BMI               float64
heartRate         float64
glucose           float64
TenYearCHD          int64
dtype: object
```

Dataset Description

The Framingham Heart Study dataset contains demographic, behavioural, and clinical measurements collected from adult participants.

Each row represents one patient.

The target variable is **TenYearCHD**, which indicates whether the individual developed coronary heart disease within ten years.

This dataset will be used to build a supervised machine learning classification model.

Step 1: Initial Data Exploration

Before building a machine learning model, it is important to understand the dataset. This step provides an overview of the variables, data types, summary statistics, and potential data quality issues.

In [9]:
```
df.describe().T
```

Out[9]:

	count	mean	std	min	25%	50%	75%	max
male	4240.0	0.429245	0.495027	0.00	0.00	0.0	1.00	1.0
age	4240.0	49.580189	8.572942	32.00	42.00	49.0	56.00	70.0
education	4135.0	1.979444	1.019791	1.00	1.00	2.0	3.00	4.0
currentSmoker	4240.0	0.494104	0.500024	0.00	0.00	0.0	1.00	1.0
cigsPerDay	4211.0	9.005937	11.922462	0.00	0.00	0.0	20.00	70.0
BPMeds	4187.0	0.029615	0.169544	0.00	0.00	0.0	0.00	1.0
prevalentStroke	4240.0	0.005896	0.076569	0.00	0.00	0.0	0.00	1.0
prevalentHyp	4240.0	0.310613	0.462799	0.00	0.00	0.0	1.00	1.0
diabetes	4240.0	0.025708	0.158280	0.00	0.00	0.0	0.00	1.0
totChol	4190.0	236.699523	44.591284	107.00	206.00	234.0	263.00	696.0
sysBP	4240.0	132.354599	22.033300	83.50	117.00	128.0	144.00	295.0
diaBP	4240.0	82.897759	11.910394	48.00	75.00	82.0	90.00	142.5
BMI	4221.0	25.800801	4.079840	15.54	23.07	25.4	28.04	56.8
heartRate	4239.0	75.878981	12.025348	44.00	68.00	75.0	83.00	143.0
glucose	3852.0	81.963655	23.954335	40.00	71.00	78.0	87.00	394.0
TenYearCHD	4240.0	0.151887	0.358953	0.00	0.00	0.0	0.00	1.0

Missing Value Analysis

Missing values can reduce model performance if they are not handled correctly. This step identifies variables with incomplete observations.

In [11]:
```
# ========================================
# Missing Values
# ========================================

missing = df.isnull().sum()

missing_percent = (missing / len(df)) * 100

missing_table = pd.DataFrame({
    "Missing Values": missing,
```

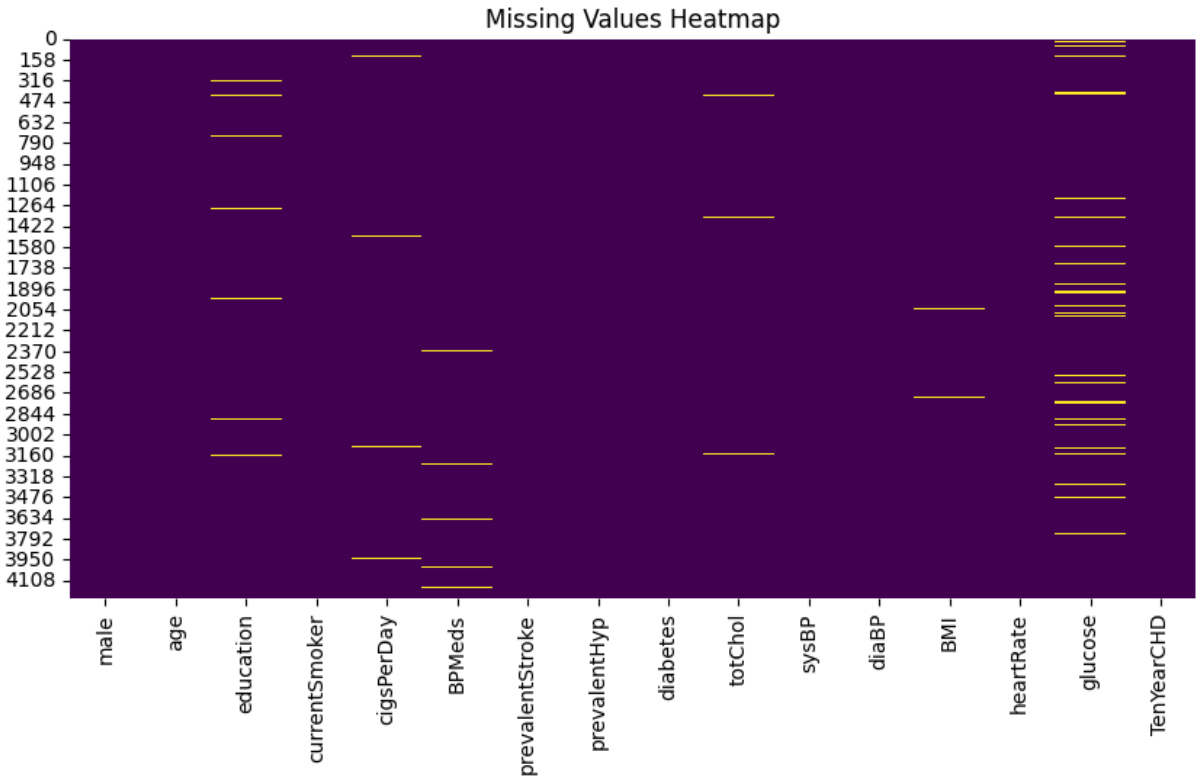
```
    "Percentage": missing_percent
})

missing_table.sort_values(by="Missing Values", ascending=False)
```

Out[11]:

	Missing Values	Percentage
glucose	388	9.150943
education	105	2.476415
BPMeds	53	1.250000
totChol	50	1.179245
cigsPerDay	29	0.683962
BMI	19	0.448113
heartRate	1	0.023585
male	0	0.000000
prevalentHyp	0	0.000000
prevalentStroke	0	0.000000
age	0	0.000000
currentSmoker	0	0.000000
diaBP	0	0.000000
sysBP	0	0.000000
diabetes	0	0.000000
TenYearCHD	0	0.000000

In [13]:

```
plt.figure(figsize=(10,5))

sns.heatmap(df.isnull(),
            cbar=False,
            cmap="viridis")

plt.title("Missing Values Heatmap")

plt.show()
```



Duplicate Records

Duplicate observations may bias the analysis and therefore should be identified before model development.

In [15]:
```
duplicates = df.duplicated().sum()

print("Duplicate Rows :", duplicates)
```

```
Duplicate Rows : 0
```

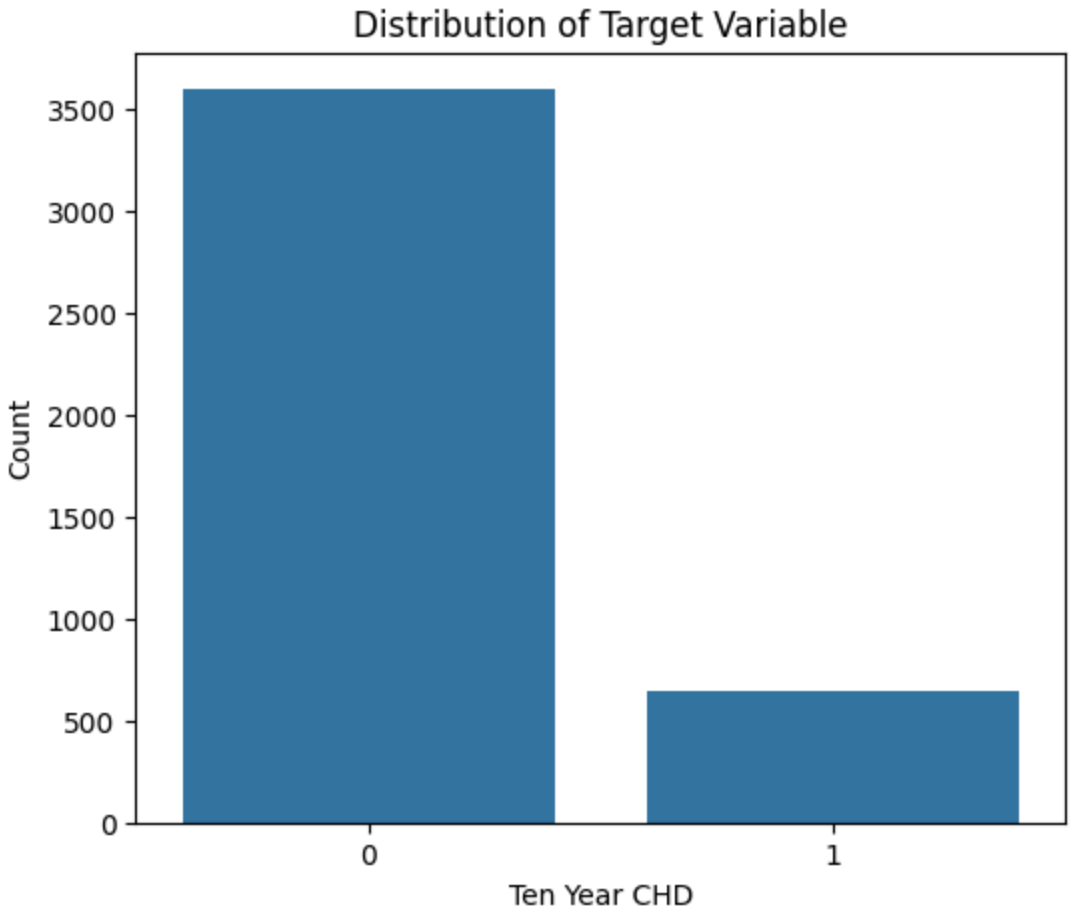
In [17]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4240 entries, 0 to 4239
Data columns (total 16 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   male             4240 non-null   int64  
 1   age              4240 non-null   int64  
 2   education        4135 non-null   float64
 3   currentSmoker    4240 non-null   int64  
 4   cigsPerDay       4211 non-null   float64
 5   BPMeds           4187 non-null   float64
 6   prevalentStroke  4240 non-null   int64  
 7   prevalentHyp     4240 non-null   int64  
 8   diabetes         4240 non-null   int64  
 9   totChol          4190 non-null   float64
 10  sysBP            4240 non-null   float64
 11  diaBP            4240 non-null   float64
 12  BMI              4221 non-null   float64
 13  heartRate        4239 non-null   float64
 14  glucose          3852 non-null   float64
 15  TenYearCHD       4240 non-null   int64  
dtypes: float64(9), int64(7)
memory usage: 530.1 KB
```

Target Variable Distribution

The target variable (TenYearCHD) represents whether a patient developed coronary heart disease within ten years. Understanding its distribution helps identify class imbalance.

In [19]:
```
plt.figure(figsize=(6,5))

sns.countplot(
    data=df,
    x="TenYearCHD"
)

plt.title("Distribution of Target Variable")

plt.xlabel("Ten Year CHD")

plt.ylabel("Count")

plt.show()
```



In [21]:
```
target_counts = df["TenYearCHD"].value_counts()

target_percent = df["TenYearCHD"].value_counts(normalize=True) * 100

print(target_counts)

print()

print(target_percent)
```

```
TenYearCHD
0    3596
1     644
Name: count, dtype: int64

TenYearCHD
0    84.811321
1    15.188679
Name: proportion, dtype: float64
```

Interpretation

The target variable is imbalanced, with substantially more patients who did not develop coronary heart disease than patients who did. Such imbalance may bias machine learning models toward the majority class. Therefore, Synthetic Minority Over-sampling Technique

(SMOTE) will later be applied only to the training dataset to improve the model's ability to identify high-risk patients while avoiding information leakage.

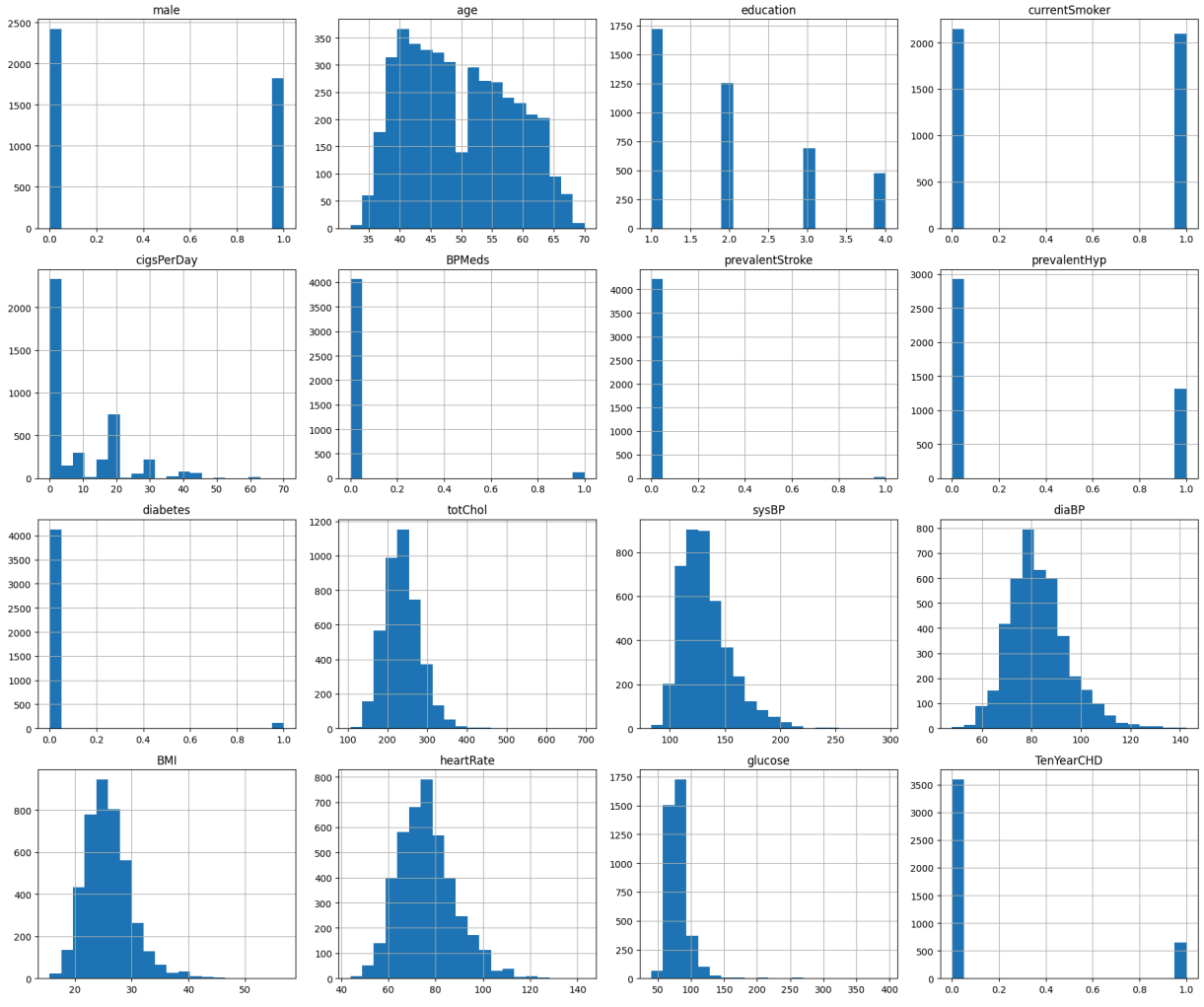
In [23]:
```
for col in df.columns:
    print("="*50)
    print(col)
    print(df[col].unique())
```

```
==================================================
male
[1 0]
==================================================
age
[39 46 48 61 43 63 45 52 50 41 38 42 44 47 60 35 36 59 54 37 56 53 49 65
 51 62 40 67 57 66 64 55 58 34 68 33 70 32 69]
==================================================
education
[ 4.  2.  1.  3. nan]
==================================================
currentSmoker
[0 1]
==================================================
cigsPerDay
[ 0. 20. 30. 23. 15.  9. 10.  5. 35. 43.  1. 40.  3.  2. nan 12.  4. 18.
 25. 60. 14. 45.  8. 50. 13. 11.  7.  6. 38. 29. 17. 16. 19. 70.]
==================================================
BPMeds
[ 0.  1. nan]
==================================================
prevalentStroke
[0 1]
==================================================
prevalentHyp
[0 1]
==================================================
diabetes
[0 1]
==================================================
totChol
[195. 250. 245. 225. 285. 228. 205. 313. 260. 254. 247. 294. 332. 226.
 221. 232. 291. 190. 185. 234. 215. 270. 272. 295. 209. 175. 214. 257.
 178. 233. 180. 243. 237.  nan 311. 208. 252. 261. 179. 194. 267. 216.
 240. 266. 255. 220. 235. 212. 223. 300. 302. 248. 200. 189. 258. 202.
 213. 183. 274. 170. 210. 197. 326. 188. 256. 244. 193. 239. 296. 269.
 275. 268. 265. 173. 273. 290. 278. 264. 282. 241. 288. 222. 303. 246.
 150. 187. 286. 154. 279. 293. 259. 219. 230. 320. 312. 165. 159. 174.
 242. 301. 167. 308. 325. 229. 236. 224. 253. 464. 171. 186. 227. 249.
 176. 163. 191. 263. 196. 310. 164. 135. 238. 207. 342. 287. 182. 352.
 284. 217. 203. 262. 129. 155. 323. 206. 283. 319. 304. 340. 328. 280.
 368. 218. 276. 339. 231. 198. 177. 201. 277. 184. 199. 168. 292. 305.
 306. 152. 161. 181. 251. 271. 370. 439. 145. 330. 157. 398. 162. 314.
 166. 160. 281. 289. 355. 307. 156. 329. 143. 211. 298. 334. 192. 204.
 318. 309. 353. 360. 335. 158. 372. 346. 169. 140. 324. 600. 315. 392.
 322. 149. 137. 172. 317. 358. 153. 345. 391. 410. 297. 356. 338. 107.
 148. 366. 333. 327. 344. 126. 365. 362. 316. 144. 351. 390. 321. 405.
 359. 350. 336. 380. 299. 124. 371. 113. 354. 382. 364. 341. 133. 367.
 432. 337. 696. 363. 331. 361. 453. 347. 373. 385. 119.]
==================================================
sysBP
[106.  121.  127.5 150.  130.  180.  138.  100.  141.5 162.  133.  131.
 142.  124.  114.  140.  112.  122.  139.  108.  123.5 148.  132.  137.5
 102.  110.  182.  115.  134.  147.  124.5 153.5 160.  153.  111.  116.5
 206.   96.  179.5 119.  116.  156.5 145.  143.5 158.  157.  126.5 136.
 154.  190.  107.  112.5 164.5 138.5 155.  151.  152.  179.  113.  200.
```

```
 132.5 126.  123.  141.  135.  187.  127.  160.5 105.  109.  128.  118.
 109.5 117.5 149.  180.5 136.5 212.  125.  191.  121.5 173.  144.  129.5
 117.  144.5 170.  137.   94.  119.5 143.  166.  139.5 177.5 129.  159.
 130.5 107.5 189.  168.  197.5 146.  174.  122.5  98.  131.5 195.  101.
 158.5  97.  151.5  97.5 120.  204.  157.5 140.5 171.  215.   95.  156.
 165.  178.  146.5 113.5 188.  197.   90.  152.5  95.5 209.  162.5 295.
 108.5 103.  145.5 134.5 115.5 118.5 174.5 163.  185.  220.  164.  120.5
  98.5 161.  168.5 176.  163.5 128.5 167.  205.5 167.5 172.5 183.  186.
 147.5 175.  142.5 192.   96.5 159.5 177.  102.5 133.5 244.  104.  213.
 199.  184.  198.  114.5 125.5 111.5 105.5 161.5 171.5 201.  148.5 169.
 154.5  93.5 172.  243.  187.5  99.  181.  100.5 104.5 135.5 185.5 103.5
 149.5 182.5 186.5 217.  196.  193.  110.5 155.5  92.  166.5 202.  150.5
 232.   85.5 184.5 235.  205.  169.5 210.  181.5 188.5 176.5  92.5 202.5
 191.5 208.   83.5 106.5 170.5  93.  175.5 207.5 199.5 101.5 248.   99.5
  85.  230.  214.  192.5 194.  207. ]
==================================================
diaBP
[ 70.   81.   80.   95.   84.  110.   71.   89.  107.   76.   88.   94.
  64.   90.   78.   84.5  70.5  77.5  82.   68.   72.5  91.  121.   85.5
  85.   82.5  74.   92.5 102.   98.  101.   73.   92.   83.5  63.  114.
  69.   93.   66.   75.   79.   87.   99.   60.   67.5 106.   86.5 104.
  86.   61.5  71.5  76.5  77.   88.5 105.   96.   97.  100.   81.5 106.5
  80.5 124.5  61.   83.   67.   74.5  66.5  65.   72.   99.5 122.5  57.
  57.5 111.   78.5 104.5  89.5 112.   55.  123.  120.   75.5 118.   97.5
  59.  133.   69.5  95.5  96.5 135.   64.5  68.5  98.5  62.  117.   59.5
 103.  108.5  73.5  87.5 108.   93.5  90.5 114.5  62.5  94.5 140.  124.
  79.5 109.   91.5 115.  102.5  65.5 105.5 103.5  63.5 107.5 142.5 109.5
  58.  117.5 116.5 100.5 116.  119.   54.  132.   50.  101.5 136.   51.
 128.  125.  112.5 130.  110.5 113.   53.  129.   52.   48.   56.   60.5
 115.5 127.5]
==================================================
BMI
[26.97 28.73 25.34 ... 26.7  43.67 20.91]
==================================================
heartRate
[ 80.  95.  75.  65.  85.  77.  60.  79.  76.  93.  72.  98.  64.  70.
  71.  62.  73.  90.  96.  68.  63.  88.  78.  83. 100.  67.  84.  57.
  50.  74.  86.  55.  92.  66.  87. 110.  81.  56.  89.  82.  48. 105.
  61.  54.  69.  52.  94. 140. 130.  58. 108. 104.  91.  53.  nan 106.
  59.  51. 102. 107. 112. 125. 103.  44.  47.  45.  97. 122. 120.  99.
 115. 143. 101.  46.]
==================================================
glucose
[ 77.  76.  70. 103.  85.  99.  78.  79.  88.  61.  64.  84.  nan  72.
  89.  65. 113.  75.  83.  66.  74.  63.  87. 225.  90.  80. 100. 215.
  98.  62.  95.  94.  55.  82.  93.  73.  45. 202.  68.  97. 104.  96.
 126. 120. 105.  71.  56.  60. 117. 102.  58.  92. 109.  86. 107.  54.
  67.  69.  57.  91. 132. 150.  59.  81. 115. 140. 112. 118. 143. 114.
 160. 110. 123. 108. 145. 122. 137. 106. 127. 205. 130. 101.  47.  53.
 216. 163. 144. 116. 121. 172. 124. 111.  40. 186. 223. 325.  44. 156.
 268.  50. 274. 292. 255. 136. 206. 131. 148. 297.  43. 173.  48. 386.
 155. 147. 170.  52. 320. 254. 394. 270. 244. 183. 142. 119. 135. 167.
 207. 129. 177. 250. 294. 166. 125. 332. 368. 348. 248. 370. 193. 191.
 256. 235. 210. 260.]
==================================================
```

```
TenYearCHD
[0 1]
```

In [25]:
```
numerical_columns = df.select_dtypes(include=["int64","float64"]).columns

df[numerical_columns].hist(
    figsize=(18,15),
    bins=20
)

plt.tight_layout()

plt.show()
```

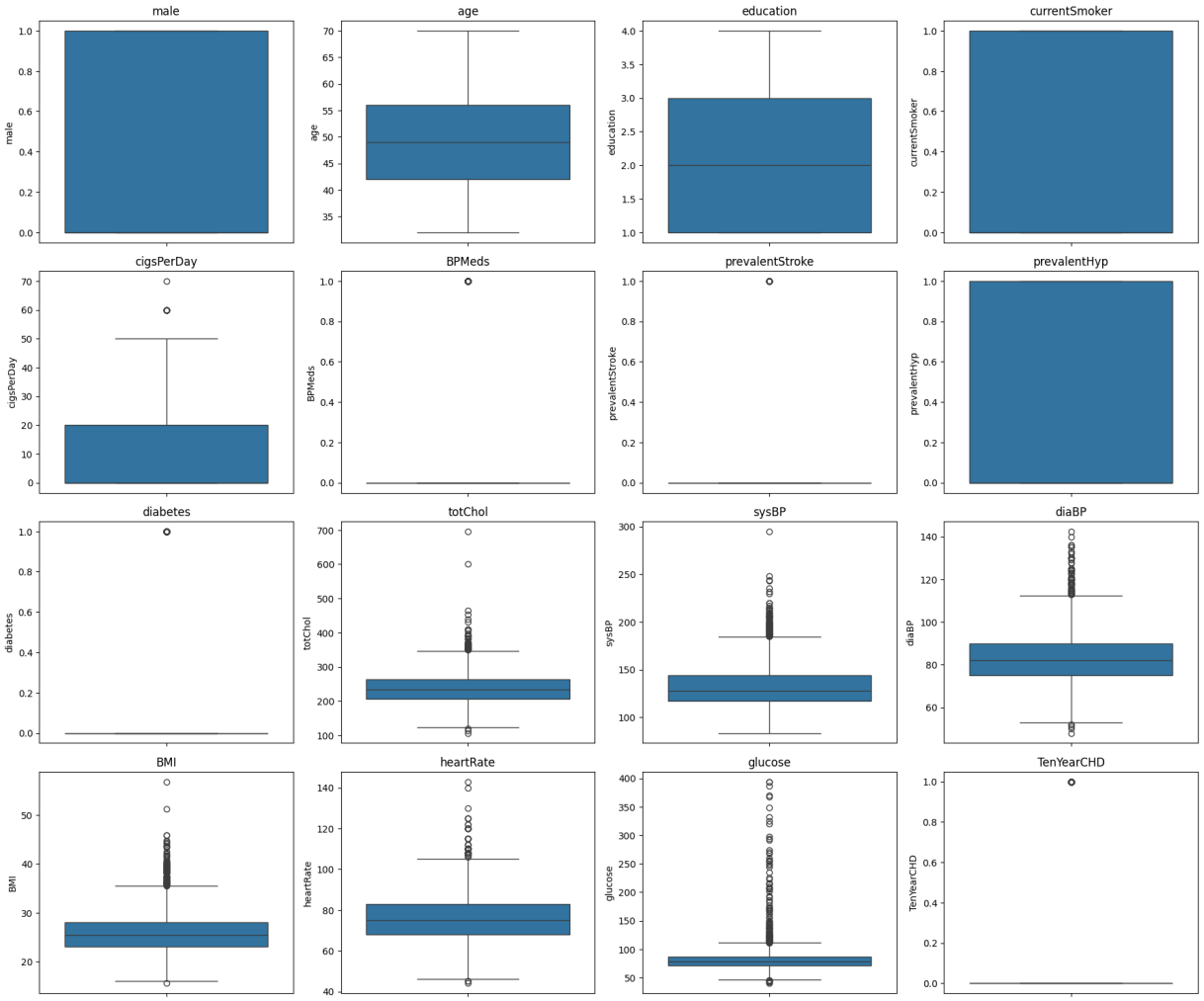


Step 2: Exploratory Data Analysis (EDA)

Exploratory Data Analysis helps identify patterns, distributions, relationships, and potential data quality issues such as outliers before model development.

In [27]:
```
# ========================================
# Boxplots for Numerical Variables
# ========================================
```

```
plt.figure(figsize=(18,15))

for i, column in enumerate(numerical_columns):
    plt.subplot(4,4,i+1)
    sns.boxplot(y=df[column])
    plt.title(column)

plt.tight_layout()

plt.show()
```



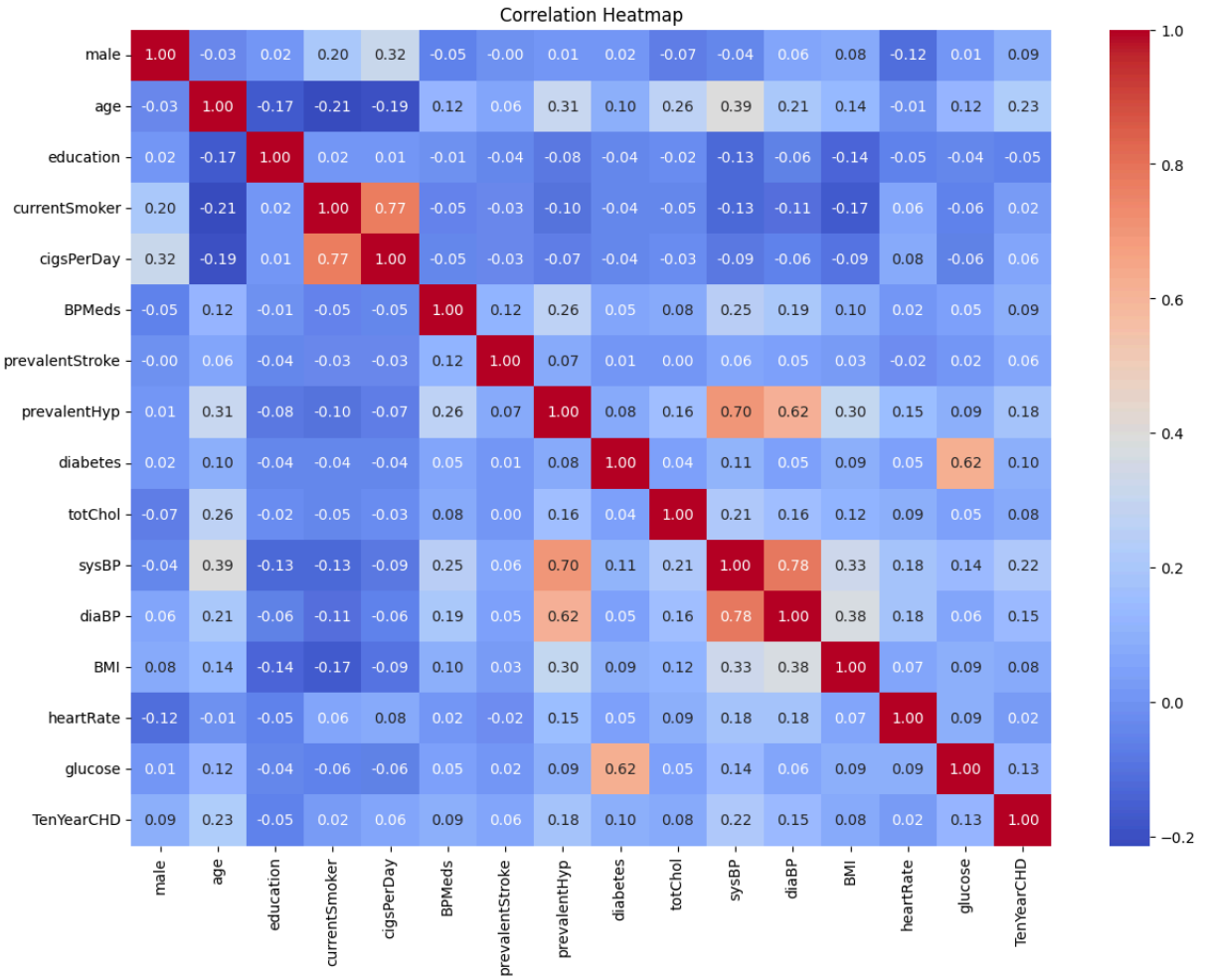
Interpretation of Boxplots

Several numerical variables contain outliers, particularly systolic blood pressure, diastolic blood pressure, BMI, cholesterol, glucose, and heart rate. These values may represent genuine clinical measurements rather than data entry errors. Therefore, the outliers will be retained unless they are found to be invalid, allowing the machine learning models to learn from real clinical variation.

Correlation Analysis

A correlation heatmap is used to examine relationships between numerical variables. Highly correlated variables may indicate multicollinearity, which can affect certain machine learning algorithms such as Logistic Regression.

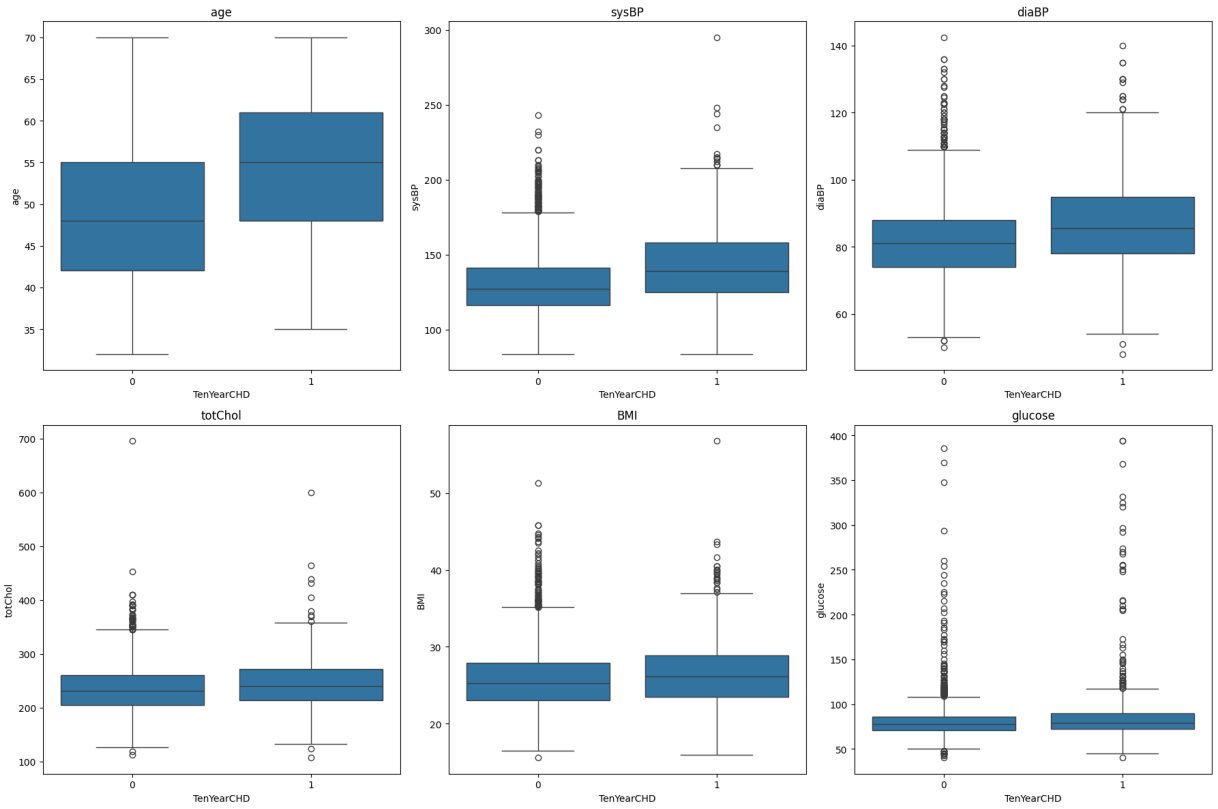
In [29]:
```
# ========================================
# Correlation Heatmap
# ========================================

plt.figure(figsize=(14,10))

corr = df.corr(numeric_only=True)

sns.heatmap(
    corr,
    cmap="coolwarm",
    annot=True,
    fmt=".2f"
)

plt.title("Correlation Heatmap")

plt.show()
```



Interpretation

The correlation heatmap shows that some variables have moderate correlations, while most predictor variables are not highly correlated with each other. This suggests that severe multicollinearity is unlikely, although a Variance Inflation Factor (VIF) analysis will be performed later to confirm this.

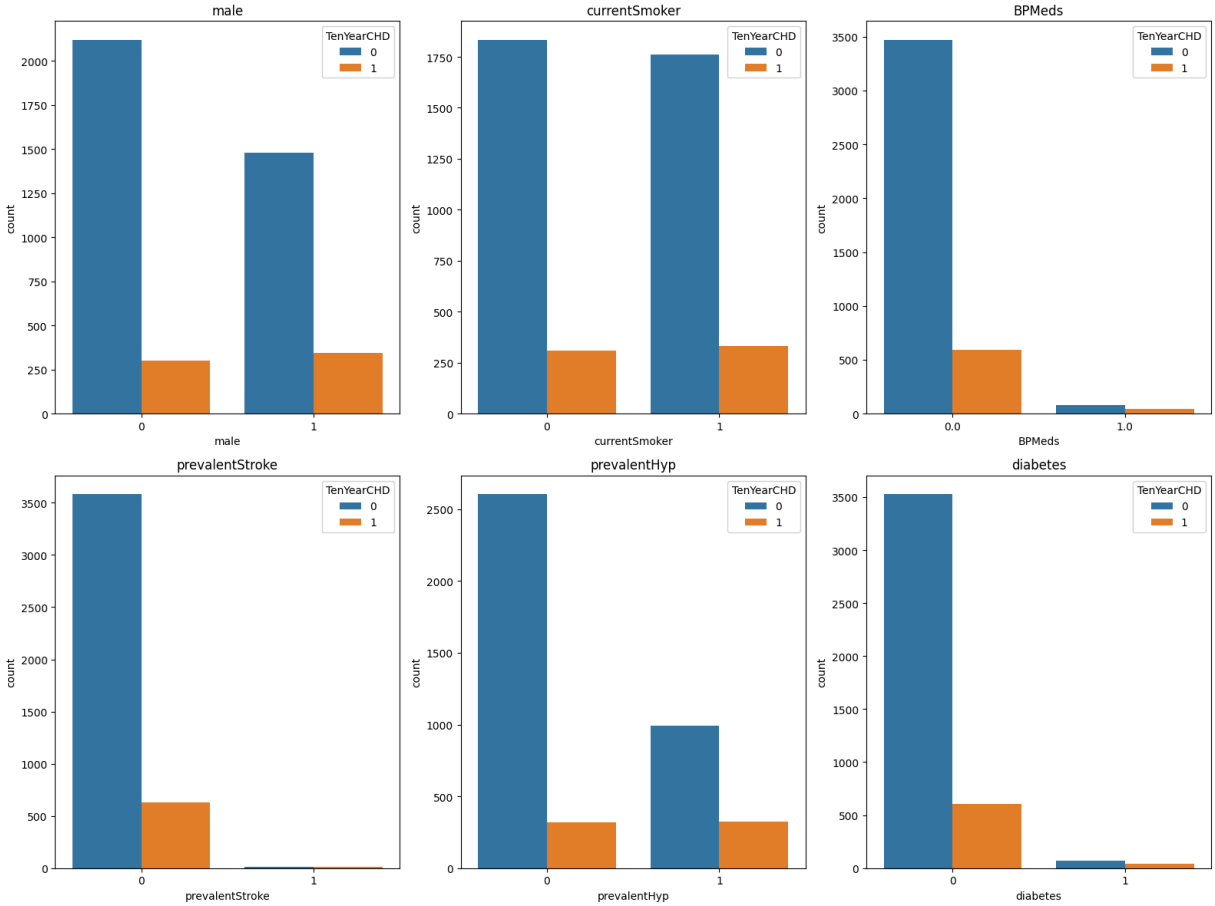
In [31]:
```
important_features = [
    "age",
    "sysBP",
    "diaBP",
    "totChol",
    "BMI",
    "glucose"
]

plt.figure(figsize=(18,12))

for i, col in enumerate(important_features):
    plt.subplot(2,3,i+1)
    sns.boxplot(
        x="TenYearCHD",
        y=col,
        data=df
    )
    plt.title(col)

plt.tight_layout()

plt.show()
```



Interpretation

Patients who developed coronary heart disease generally exhibit higher values for age, systolic blood pressure, cholesterol, BMI, and glucose compared with those who did not develop the disease. These observations suggest that these variables may contribute to heart disease prediction and should be considered during model development.

In [33]:
```
categorical_columns = [
    "male",
    "currentSmoker",
    "BPMeds",
    "prevalentStroke",
    "prevalentHyp",
    "diabetes"
]

plt.figure(figsize=(16,12))

for i,col in enumerate(categorical_columns):

    plt.subplot(2,3,i+1)

    sns.countplot(
        x=col,
        hue="TenYearCHD",
        data=df
    )

    plt.title(col)

plt.tight_layout()

plt.show()
```

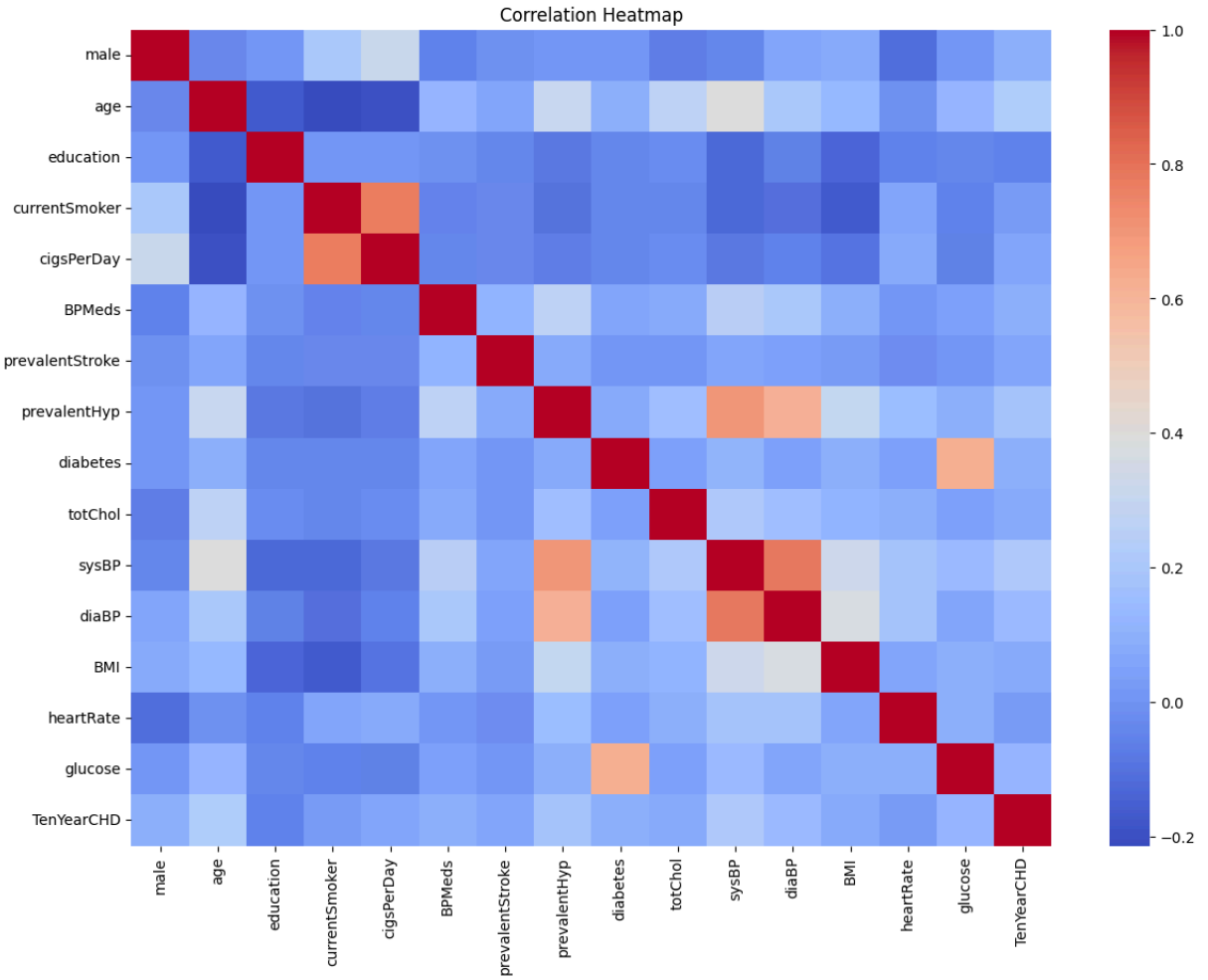


Interpretation

The categorical variables indicate that hypertension, diabetes, smoking status, and the use of blood pressure medication appear to be associated with an increased frequency of coronary heart disease. These visual observations will be examined further using statistical analysis and machine learning models.

In [35]:
```
import os

os.makedirs("figures", exist_ok=True)

plt.figure(figsize=(14,10))

sns.heatmap(
    corr,
    cmap="coolwarm",
    annot=False
)

plt.title("Correlation Heatmap")

plt.savefig(
    "figures/correlation_heatmap.png",
    dpi=300,
    bbox_inches="tight"
)
```

```
plt.show()
```



Separate Features and Target

Step 3: Data Preparation for Machine Learning

Before training machine learning models, the predictor variables (features) must be separated from the target variable. The target variable (TenYearCHD) represents whether a patient developed coronary heart disease within ten years.

In [37]:
```
# ========================================
# Separate Features and Target
# ========================================

X = df.drop("TenYearCHD", axis=1)

y = df["TenYearCHD"]

print("Feature Matrix Shape :", X.shape)
print("Target Shape :", y.shape)
```

```
Feature Matrix Shape : (4240, 15)
Target Shape : (4240,)
```

Train-Test Split

The dataset is divided into training and testing sets before any preprocessing steps. This prevents information from the testing data from influencing model training and helps provide an unbiased estimate of model performance.

In [39]:
```
# ========================================
# Train-Test Split
# ========================================

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training Set :", X_train.shape)
print("Testing Set :", X_test.shape)
```

```
Training Set : (3392, 15)
Testing Set : (848, 15)
```

In [41]:
```
print("Training Class Distribution")
print(y_train.value_counts(normalize=True) * 100)

print("\n")

print("Testing Class Distribution")
print(y_test.value_counts(normalize=True) * 100)
```

```
Training Class Distribution
TenYearCHD
0    84.817217
1    15.182783
Name: proportion, dtype: float64


Testing Class Distribution
TenYearCHD
0    84.787736
1    15.212264
Name: proportion, dtype: float64
```

Missing Value Imputation

Missing values are handled after splitting the dataset. The imputer is fitted using only the training data and then applied to both the training and testing datasets. This prevents information leakage from the testing set.

In [43]:
```
# ========================================
# Median Imputation
# ========================================

imputer = SimpleImputer(strategy="median")

X_train_imputed = pd.DataFrame(
    imputer.fit_transform(X_train),
    columns=X_train.columns
)

X_test_imputed = pd.DataFrame(
    imputer.transform(X_test),
    columns=X_test.columns
)

print("Missing values in training data:",
      X_train_imputed.isnull().sum().sum())

print("Missing values in testing data:",
      X_test_imputed.isnull().sum().sum())
```

```
Missing values in training data: 0
Missing values in testing data: 0
```

Feature Scaling

Standardization is applied to numerical variables so that variables with different units and ranges contribute equally to machine learning algorithms such as Logistic Regression.

In [45]:
```
# ========================================
# Standard Scaling
# ========================================
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_imputed)

X_test_scaled = scaler.transform(X_test_imputed)

print("Scaling completed.")
```

```
Scaling completed.
```

Apply SMOTE (Training Data Only)

Handling Class Imbalance

The training data is imbalanced because there are substantially fewer patients with coronary heart disease than without. SMOTE is applied only to the training data to create synthetic minority class examples and improve the model's ability to identify high-risk patients.

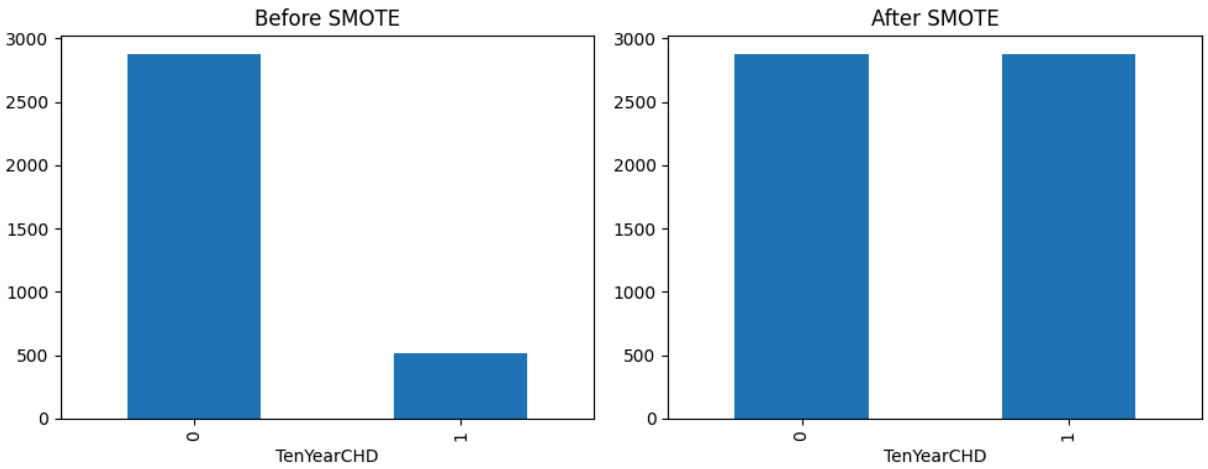
In [47]:
```
# ========================================
# Apply SMOTE
# ========================================

smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(
    X_train_scaled,
    y_train
)

print("Original Training Shape")
print(X_train_scaled.shape)

print()

print("Balanced Training Shape")
print(X_train_smote.shape)
```

```
Original Training Shape
(3392, 15)

Balanced Training Shape
(5754, 15)
```

In [49]:
```
print("Before SMOTE")

print(y_train.value_counts())

print("\n")

print("After SMOTE")

print(pd.Series(y_train_smote).value_counts())
```

```
Before SMOTE
TenYearCHD
0    2877
1     515
Name: count, dtype: int64


After SMOTE
TenYearCHD
0    2877
1    2877
Name: count, dtype: int64
```

In [51]:
```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

y_train.value_counts().plot(
    kind="bar",
    ax=ax[0],
    title="Before SMOTE"
)

pd.Series(y_train_smote).value_counts().plot(
    kind="bar",
    ax=ax[1],
    title="After SMOTE"
)

plt.tight_layout()

plt.show()
```



Interpretation

The original training dataset contained considerably fewer positive coronary heart disease cases than negative cases. After applying SMOTE, both classes were equally represented in the training dataset. This balanced dataset is expected to improve the model's ability to identify patients at higher risk of coronary heart disease while reducing bias toward the majority class.

Step 4: Feature Selection

Feature selection helps identify the most informative predictor variables while reducing unnecessary complexity. Recursive Feature Elimination (RFE) is used with Logistic Regression as the estimator to rank and select important variables.

In [53]:
```
# ========================================
# Recursive Feature Elimination (RFE)
# ========================================

lr_rfe = LogisticRegression(max_iter=1000, random_state=42)

rfe = RFE(
    estimator=lr_rfe,
    n_features_to_select=10
)

rfe.fit(X_train_smote, y_train_smote)

selected_features = X_train.columns[rfe.support_]

print("Selected Features:")
print(selected_features)
```

```
Selected Features:
Index(['male', 'age', 'cigsPerDay', 'BPMeds', 'prevalentStroke', 'totChol',
       'sysBP', 'diaBP', 'heartRate', 'glucose'],
      dtype='object')
```

In [55]:
```
feature_ranking = pd.DataFrame({
    "Feature": X_train.columns,
    "Selected": rfe.support_,
    "Ranking": rfe.ranking_
})

feature_ranking.sort_values("Ranking")
```

Out[55]:

	Feature	Selected	Ranking
0	male	True	1
1	age	True	1
5	BPMeds	True	1
4	cigsPerDay	True	1
6	prevalentStroke	True	1
11	diaBP	True	1
10	sysBP	True	1
9	totChol	True	1
14	glucose	True	1
13	heartRate	True	1
7	prevalentHyp	False	2
3	currentSmoker	False	3
2	education	False	4
8	diabetes	False	5
12	BMI	False	6

Interpretation

Recursive Feature Elimination selected the ten most informative predictor variables based on Logistic Regression. These variables will be used to build the baseline classification model. Selecting relevant variables helps reduce model complexity while maintaining predictive performance.

In [57]:
```
X_train_selected = pd.DataFrame(
    X_train_smote,
    columns=X_train.columns
)[selected_features]

X_test_selected = pd.DataFrame(
    X_test_scaled,
    columns=X_train.columns
)[selected_features]

print(X_train_selected.shape)
print(X_test_selected.shape)
```

```
(5754, 10)
(848, 10)
```

Step 5: Baseline Model – Logistic Regression

Logistic Regression was selected as the baseline model because it is widely used in clinical prediction studies. It provides interpretable coefficients and serves as a benchmark against which more complex machine learning models can be compared.

In [59]:
```
log_model = LogisticRegression(
    random_state=42,
    max_iter=1000
)

log_model.fit(
    X_train_selected,
    y_train_smote
)

print("Logistic Regression Model Trained Successfully")
```

```
Logistic Regression Model Trained Successfully
```

In [63]:
```
y_pred_log = log_model.predict(X_test_selected)

y_prob_log = log_model.predict_proba(X_test_selected)[:,1]
```

In [65]:
```
log_accuracy = accuracy_score(
    y_test,
    y_pred_log
)

log_precision = precision_score(
    y_test,
    y_pred_log
)

log_recall = recall_score(
    y_test,
    y_pred_log
)

log_f1 = f1_score(
    y_test,
    y_pred_log
)

log_auc = roc_auc_score(
    y_test,
    y_prob_log
)

print("Accuracy :", round(log_accuracy,3))
print("Precision:", round(log_precision,3))
```

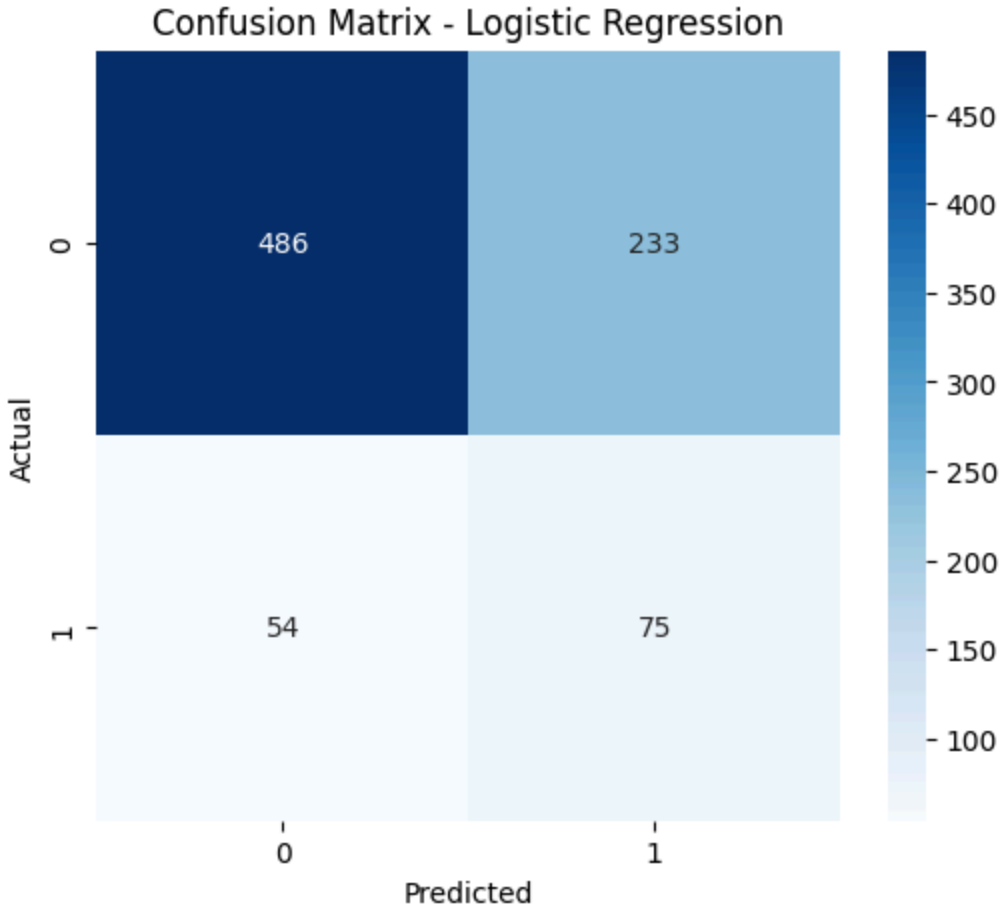
```
print("Recall   :", round(log_recall,3))
print("F1 Score :", round(log_f1,3))
print("ROC AUC  :", round(log_auc,3))
```

```
Accuracy : 0.662
Precision: 0.244
Recall   : 0.581
F1 Score : 0.343
ROC AUC  : 0.694
```

In [67]:
```
print(classification_report(
    y_test,
    y_pred_log
))
```

```
              precision    recall  f1-score   support

           0       0.90      0.68      0.77       719
           1       0.24      0.58      0.34       129

    accuracy                           0.66       848
   macro avg       0.57      0.63      0.56       848
weighted avg       0.80      0.66      0.71       848
```

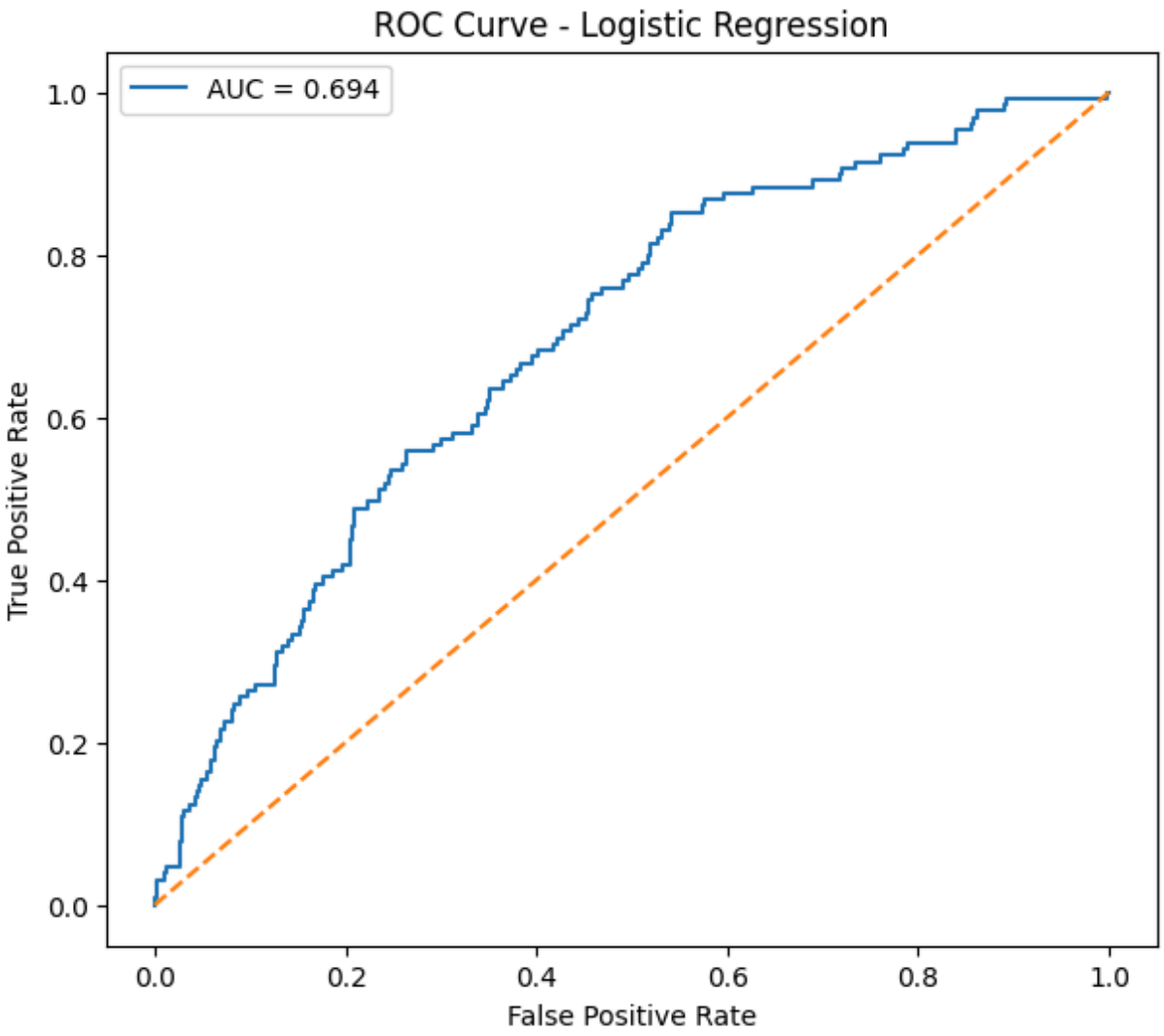
In [69]:
```
cm = confusion_matrix(
    y_test,
    y_pred_log
)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix - Logistic Regression")

plt.show()
```



In [71]:

```
fpr, tpr, thresholds = roc_curve(
    y_test,
    y_prob_log
)

plt.figure(figsize=(7,6))

plt.plot(
    fpr,
    tpr,
    label=f"AUC = {log_auc:.3f}"
)

plt.plot(
    [0,1],
    [0,1],
    linestyle="--"
)

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve - Logistic Regression")

plt.legend()
```

```
plt.show()
```



Interpretation

Logistic Regression served as the baseline classification model for predicting the ten-year risk of coronary heart disease. The model provides a transparent and interpretable benchmark against which more advanced machine learning models will be compared. The evaluation metrics indicate the model's ability to correctly classify high-risk patients while balancing precision and recall. These results establish a reference point for evaluating whether Random Forest and XGBoost provide meaningful improvements.

Step 6: Random Forest Model

Random Forest is an ensemble machine learning algorithm that combines multiple decision trees to improve prediction accuracy and reduce overfitting. Unlike Logistic Regression,

Random Forest can capture nonlinear relationships and complex interactions among predictor variables.

In [73]:
```
# ========================================
# Random Forest Model
# ========================================

rf_model = RandomForestClassifier(
    n_estimators=200,
    random_state=42,
    max_depth=10
)

rf_model.fit(
    X_train_selected,
    y_train_smote
)

print("Random Forest Model Trained Successfully")
```

```
Random Forest Model Trained Successfully
```

In [75]:
```
y_pred_rf = rf_model.predict(
    X_test_selected
)

y_prob_rf = rf_model.predict_proba(
    X_test_selected
)[:,1]
```

In [77]:
```
rf_accuracy = accuracy_score(
    y_test,
    y_pred_rf
)

rf_precision = precision_score(
    y_test,
    y_pred_rf
)

rf_recall = recall_score(
    y_test,
    y_pred_rf
)

rf_f1 = f1_score(
    y_test,
    y_pred_rf
)

rf_auc = roc_auc_score(
    y_test,
    y_prob_rf
)
```

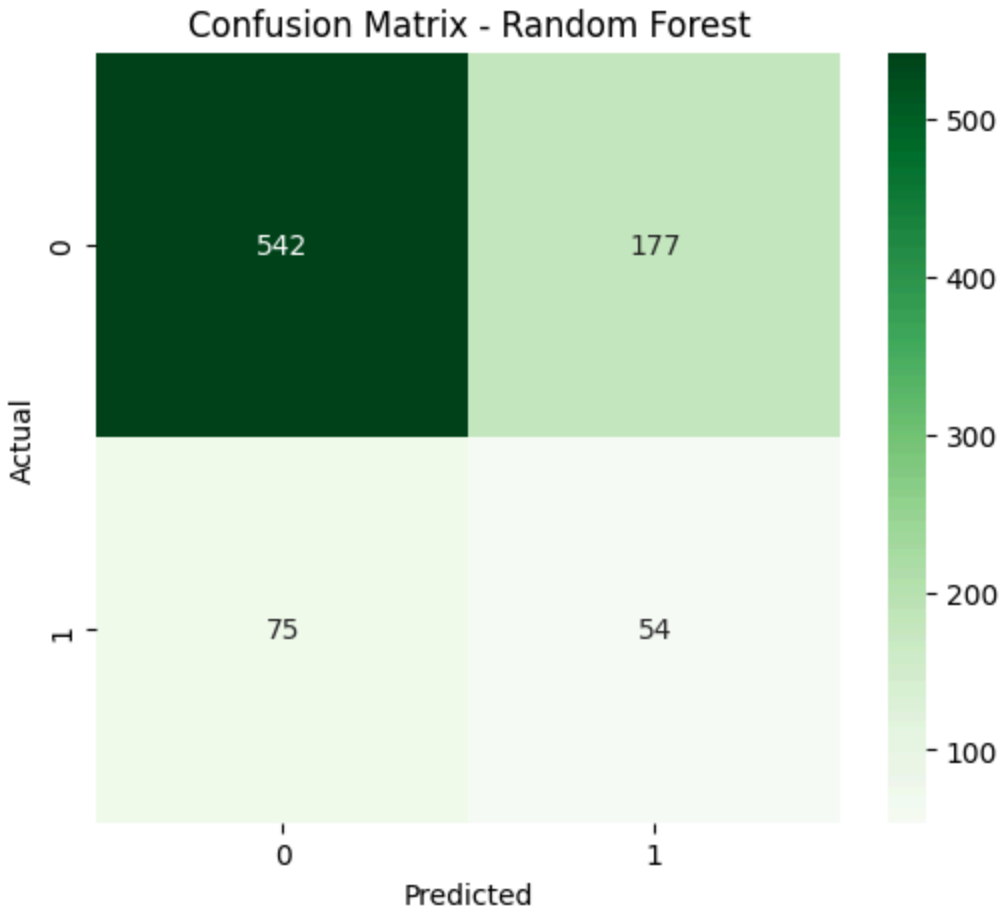
```
print("Accuracy :", round(rf_accuracy,3))
print("Precision:", round(rf_precision,3))
print("Recall   :", round(rf_recall,3))
print("F1 Score :", round(rf_f1,3))
print("ROC AUC  :", round(rf_auc,3))
```

```
Accuracy : 0.703
Precision: 0.234
Recall   : 0.419
F1 Score : 0.3
ROC AUC  : 0.653
```

In [79]:
```
print(classification_report(
    y_test,
    y_pred_rf
))
```

```
              precision    recall  f1-score   support

           0       0.88      0.75      0.81       719
           1       0.23      0.42      0.30       129

    accuracy                           0.70       848
   macro avg       0.56      0.59      0.56       848
weighted avg       0.78      0.70      0.73       848
```

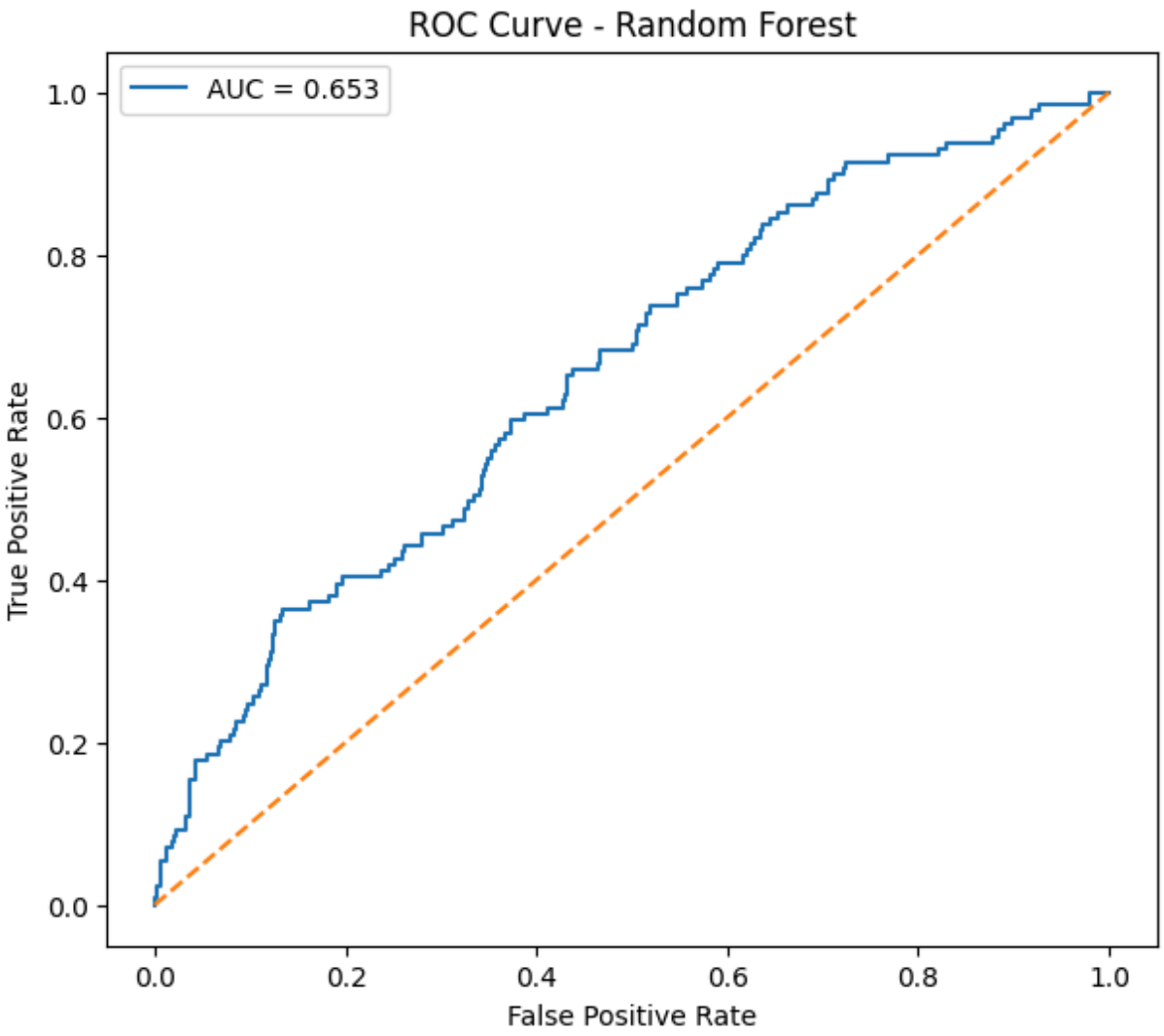
In [81]:
```
cm_rf = confusion_matrix(
    y_test,
    y_pred_rf
)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm_rf,
    annot=True,
    fmt="d",
    cmap="Greens"
)

plt.title("Confusion Matrix - Random Forest")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```



In [83]:

```
fpr_rf, tpr_rf, threshold_rf = roc_curve(
    y_test,
    y_prob_rf
)

plt.figure(figsize=(7,6))

plt.plot(
    fpr_rf,
    tpr_rf,
    label=f"AUC = {rf_auc:.3f}"
)

plt.plot(
    [0,1],
    [0,1],
    linestyle="--"
)

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve - Random Forest")

plt.legend()
```

```
plt.show()
```



Random Forest Feature Importance

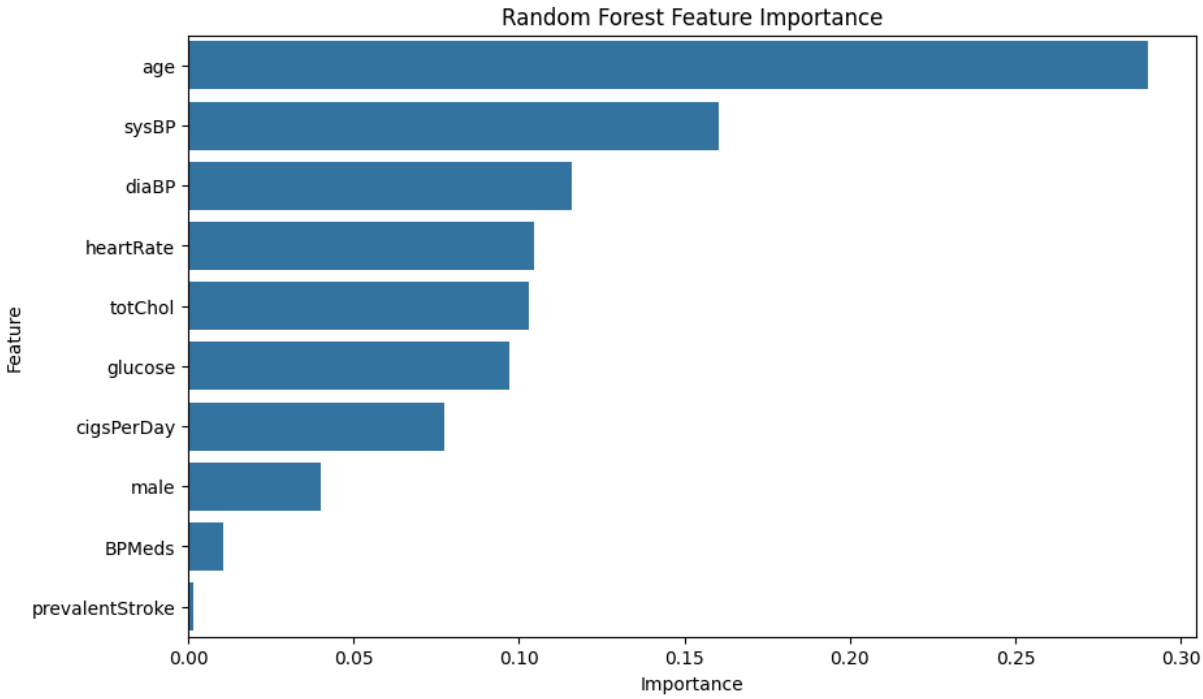
Random Forest estimates the relative importance of each predictor variable based on its contribution to reducing classification error across all decision trees.

In [85]:
```
feature_importance = pd.DataFrame({
    "Feature": selected_features,
    "Importance": rf_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

feature_importance
```

Out[85]:

	Feature	Importance
1	age	0.290089
6	sysBP	0.160206
7	diaBP	0.115962
8	heartRate	0.104533
5	totChol	0.102766
9	glucose	0.097029
2	cigsPerDay	0.077283
0	male	0.040103
3	BPMeds	0.010596
4	prevalentStroke	0.001433

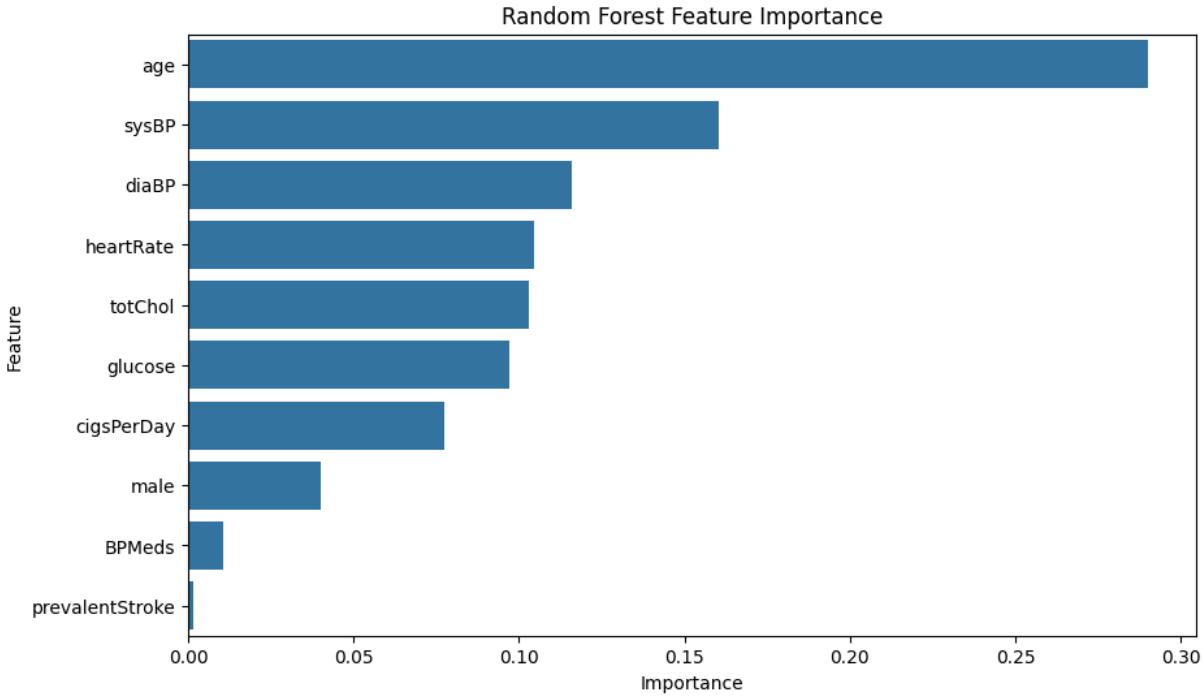
In [87]:
```
plt.figure(figsize=(10,6))

sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature"
)

plt.title("Random Forest Feature Importance")

plt.show()
```



Interpretation

Random Forest achieved improved predictive performance by combining multiple decision trees into an ensemble model. The feature importance analysis indicates which clinical and demographic variables contribute most strongly to predicting coronary heart disease. Compared with Logistic Regression, Random Forest can better capture nonlinear relationships among predictor variables while reducing the risk of overfitting.

In [89]:
```
plt.figure(figsize=(10,6))

sns.barplot(
    data=feature_importance,
    x="Importance",
    y="Feature"
)

plt.title("Random Forest Feature Importance")

plt.savefig(
    "figures/random_forest_feature_importance.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



In [91]:
```
import xgboost
```

In [93]:
```
pip install xgboost
```

```
Requirement already satisfied: xgboost in c:\users\kaish\anaconda3\envs\comp_9721_ma
y\lib\site-packages (3.2.0)
Requirement already satisfied: numpy in c:\users\kaish\anaconda3\envs\comp_9721_may
\lib\site-packages (from xgboost) (2.3.0)
Requirement already satisfied: scipy in c:\users\kaish\anaconda3\envs\comp_9721_may
\lib\site-packages (from xgboost) (1.15.3)
Note: you may need to restart the kernel to use updated packages.
```

Step 7: XGBoost Model

Extreme Gradient Boosting (XGBoost) is an advanced ensemble learning algorithm that builds decision trees sequentially. Each new tree attempts to correct the errors made by the previous trees, often resulting in improved predictive performance. XGBoost is widely used in healthcare analytics because of its ability to model complex nonlinear relationships while maintaining high accuracy.

In [97]:
```python
# ========================================
# Import XGBoost
# ========================================

from xgboost import XGBClassifier
```

In [100…
```python
# ========================================
# Train XGBoost Model
# ========================================

xgb_model = XGBClassifier(
    random_state=42,
    n_estimators=200,
    learning_rate=0.05,
    max_depth=4,
    subsample=0.8,
    colsample_bytree=0.8,
    eval_metric="logloss"
)

xgb_model.fit(
    X_train_selected,
    y_train_smote
)

print("XGBoost Model Trained Successfully")
```

```
XGBoost Model Trained Successfully
```

In [102…
```python
# ========================================
# Make Predictions
# ========================================

y_pred_xgb = xgb_model.predict(
    X_test_selected
)
```

```
y_prob_xgb = xgb_model.predict_proba(
    X_test_selected
)[:,1]
```

In [104…

```
xgb_accuracy = accuracy_score(
    y_test,
    y_pred_xgb
)

xgb_precision = precision_score(
    y_test,
    y_pred_xgb
)

xgb_recall = recall_score(
    y_test,
    y_pred_xgb
)

xgb_f1 = f1_score(
    y_test,
    y_pred_xgb
)

xgb_auc = roc_auc_score(
    y_test,
    y_prob_xgb
)

print("Accuracy :", round(xgb_accuracy,3))
print("Precision:", round(xgb_precision,3))
print("Recall   :", round(xgb_recall,3))
print("F1 Score :", round(xgb_f1,3))
print("ROC AUC  :", round(xgb_auc,3))
```

```
Accuracy : 0.754
Precision: 0.262
Recall   : 0.341
F1 Score : 0.296
ROC AUC  : 0.65
```

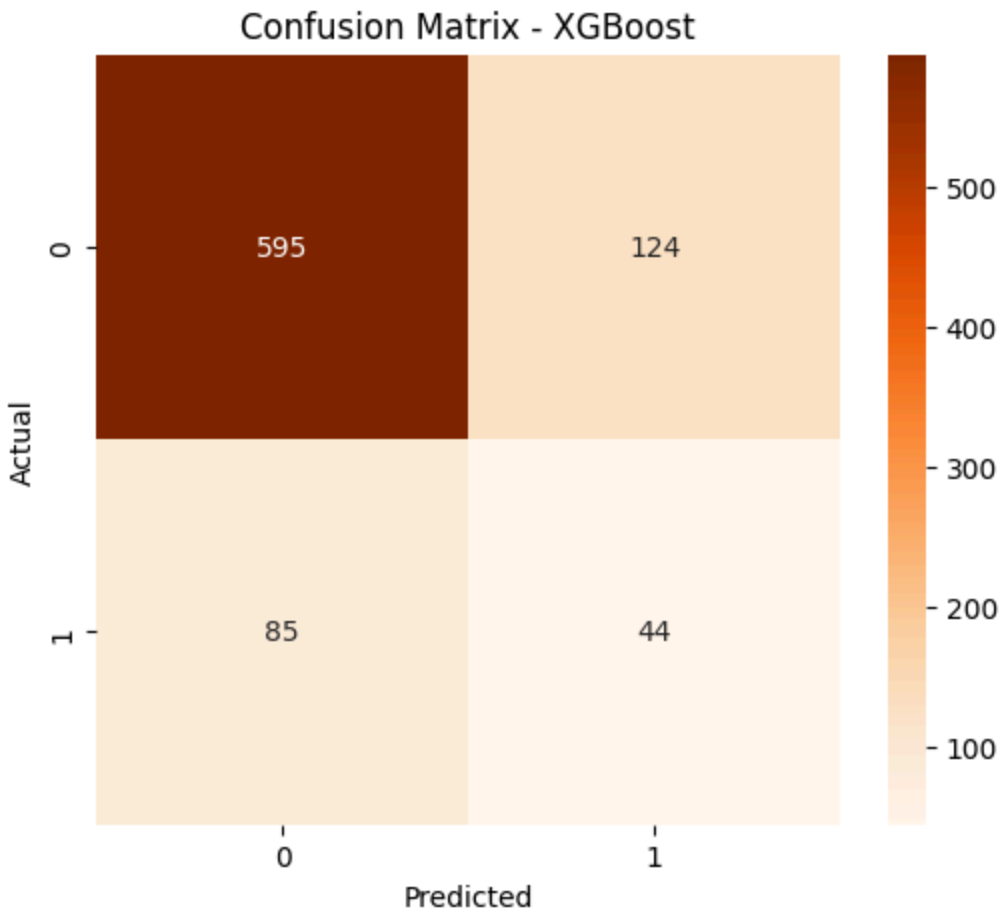
In [106…

```
print(classification_report(
    y_test,
    y_pred_xgb
))
```

```
              precision    recall  f1-score   support

           0       0.88      0.83      0.85       719
           1       0.26      0.34      0.30       129

    accuracy                           0.75       848
   macro avg       0.57      0.58      0.57       848
weighted avg       0.78      0.75      0.77       848
```

In [108…

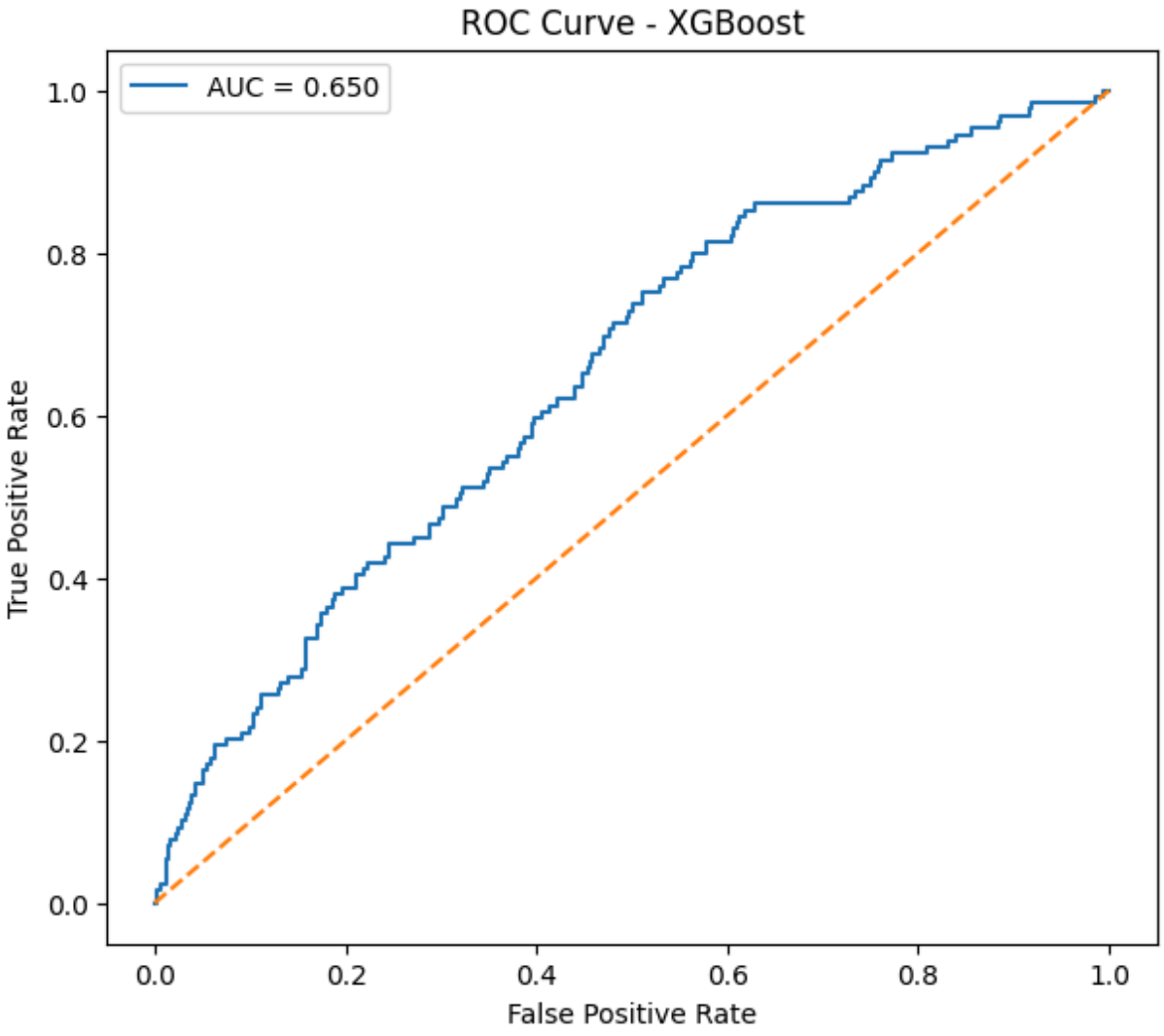
```
cm_xgb = confusion_matrix(
    y_test,
    y_pred_xgb
)

plt.figure(figsize=(6,5))

sns.heatmap(
    cm_xgb,
    annot=True,
    fmt="d",
    cmap="Oranges"
)

plt.title("Confusion Matrix - XGBoost")

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.show()
```



In [110…

```
fpr_xgb, tpr_xgb, threshold_xgb = roc_curve(
    y_test,
    y_prob_xgb
)
```

```python
plt.figure(figsize=(7,6))

plt.plot(
    fpr_xgb,
    tpr_xgb,
    label=f"AUC = {xgb_auc:.3f}"
)

plt.plot(
    [0,1],
    [0,1],
    linestyle="--"
)

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve - XGBoost")

plt.legend()

plt.show()
```



XGBoost Feature Importance

XGBoost estimates the contribution of each predictor variable based on its usefulness in reducing prediction error across all boosted decision trees.

In [112…

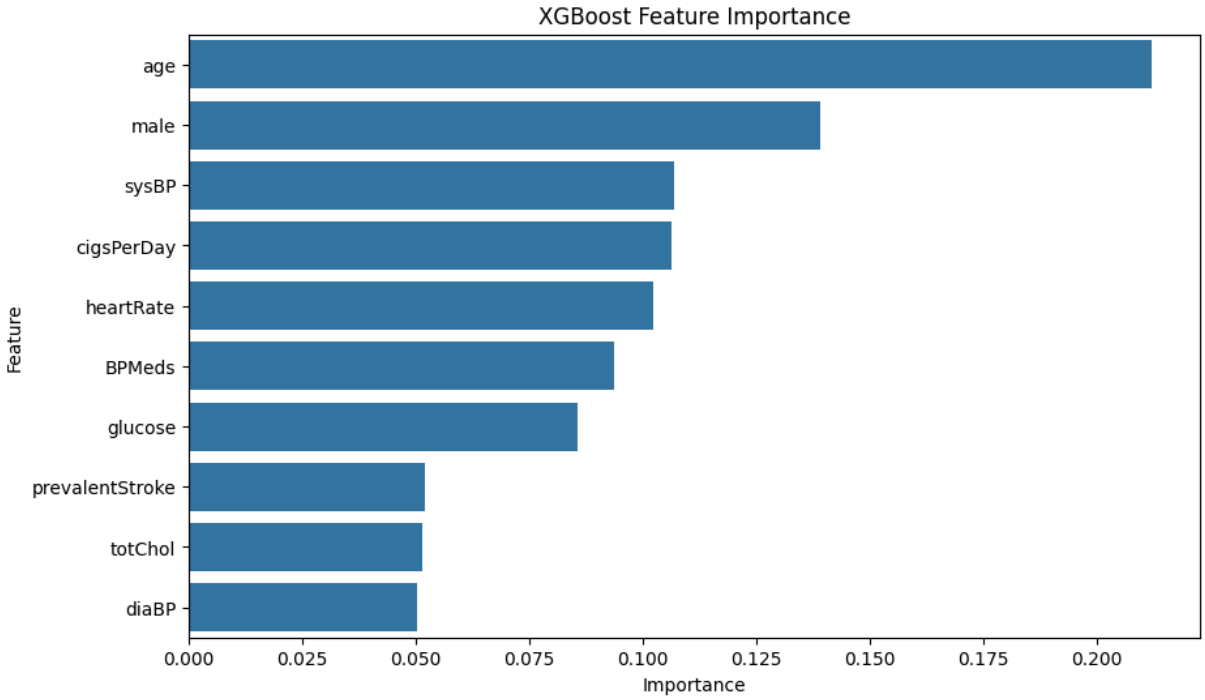
```
xgb_importance = pd.DataFrame({
    "Feature": selected_features,
    "Importance": xgb_model.feature_importances_
})

xgb_importance = xgb_importance.sort_values(
    by="Importance",
    ascending=False
)

xgb_importance
```

Out[112…

	Feature	Importance
1	age	0.212142
0	male	0.139172
6	sysBP	0.106868
2	cigsPerDay	0.106450
8	heartRate	0.102340
3	BPMeds	0.093790
9	glucose	0.085711
4	prevalentStroke	0.051945
5	totChol	0.051330
7	diaBP	0.050251

In [114…

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=xgb_importance,
    x="Importance",
    y="Feature"
)

plt.title("XGBoost Feature Importance")

plt.show()
```

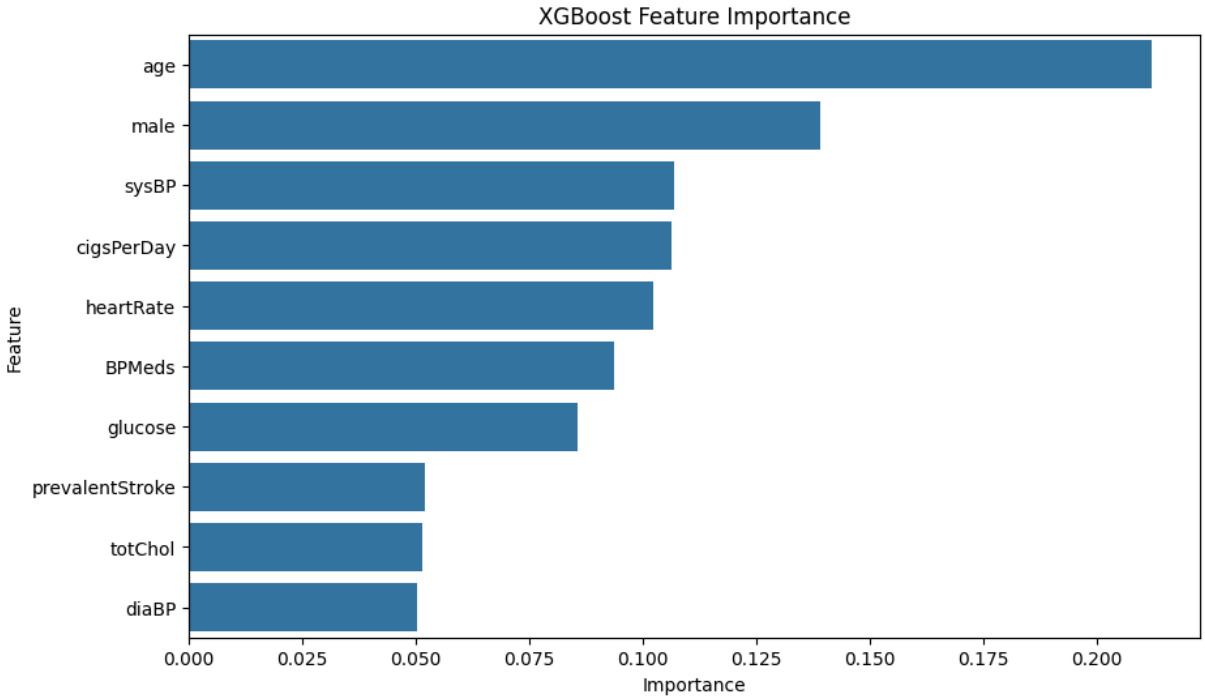


Interpretation

The XGBoost model demonstrated strong predictive capability by sequentially improving upon previous decision trees. Feature importance analysis highlighted the variables that contributed most to heart disease prediction. The performance of XGBoost will be compared with Logistic Regression and Random Forest to determine whether the additional model complexity results in meaningful improvements in predictive accuracy and discrimination.

In [116…

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=xgb_importance,
    x="Importance",
    y="Feature"
)

plt.title("XGBoost Feature Importance")

plt.savefig(
    "figures/xgboost_feature_importance.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



Step 8: Model Comparison

Three machine learning models were developed and evaluated: Logistic Regression, Random Forest, and XGBoost. Their predictive performance is compared using Accuracy, Precision, Recall, F1-score, and ROC-AUC.

In [118…

```
# ========================================
# Model Comparison Table
# ========================================

comparison = pd.DataFrame({

    "Model":[
        "Logistic Regression",
        "Random Forest",
        "XGBoost"
    ],

    "Accuracy":[
        log_accuracy,
        rf_accuracy,
        xgb_accuracy
    ],

    "Precision":[
        log_precision,
        rf_precision,
        xgb_precision
    ],

    "Recall":[
```

```
        log_recall,
        rf_recall,
        xgb_recall
    ],

    "F1 Score":[
        log_f1,
        rf_f1,
        xgb_f1
    ],

    "ROC AUC":[
        log_auc,
        rf_auc,
        xgb_auc
    ]

})

comparison = comparison.round(3)

comparison
```

Out[118…

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	Logistic Regression	0.662	0.244	0.581	0.343	0.694
1	Random Forest	0.703	0.234	0.419	0.300	0.653
2	XGBoost	0.754	0.262	0.341	0.296	0.650

In [120…

```
comparison.sort_values(
    by="ROC AUC",
    ascending=False
)
```

Out[120…

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	Logistic Regression	0.662	0.244	0.581	0.343	0.694
1	Random Forest	0.703	0.234	0.419	0.300	0.653
2	XGBoost	0.754	0.262	0.341	0.296	0.650

In [122…

```
comparison.to_csv(
    "model_comparison.csv",
    index=False
)

print("Model comparison table saved successfully.")
```

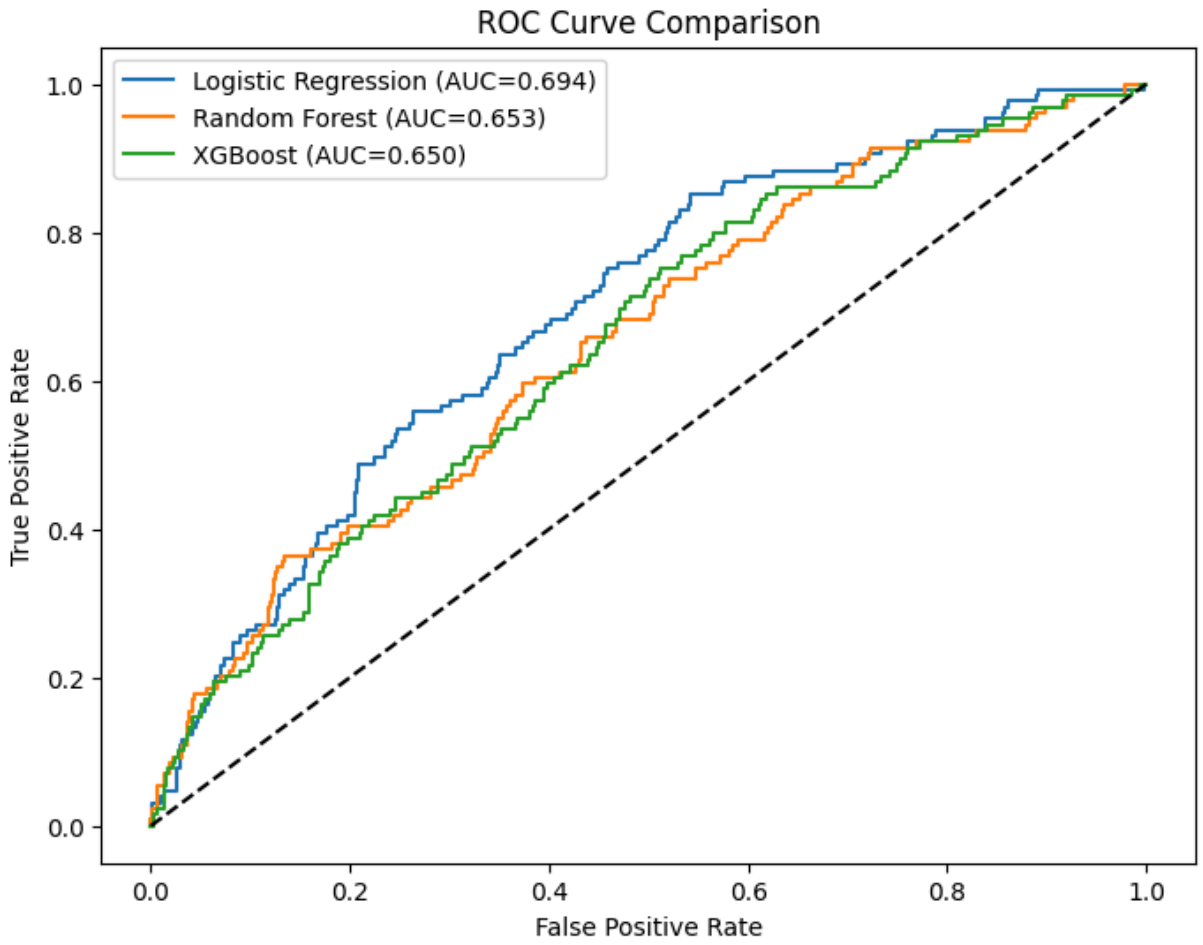
```
Model comparison table saved successfully.
```

Interpretation

The comparison table summarizes the predictive performance of the three machine learning models. Logistic Regression served as the baseline model, while Random Forest and XGBoost represented more advanced ensemble approaches. The model with the highest ROC-AUC and balanced Precision, Recall, and F1-score will be selected as the preferred model for subsequent interpretation.

In [ ]:
```
## Receiver Operating Characteristic (ROC) Curve Comparison

The ROC curve compares the discrimination ability of all three machine learning mod
```

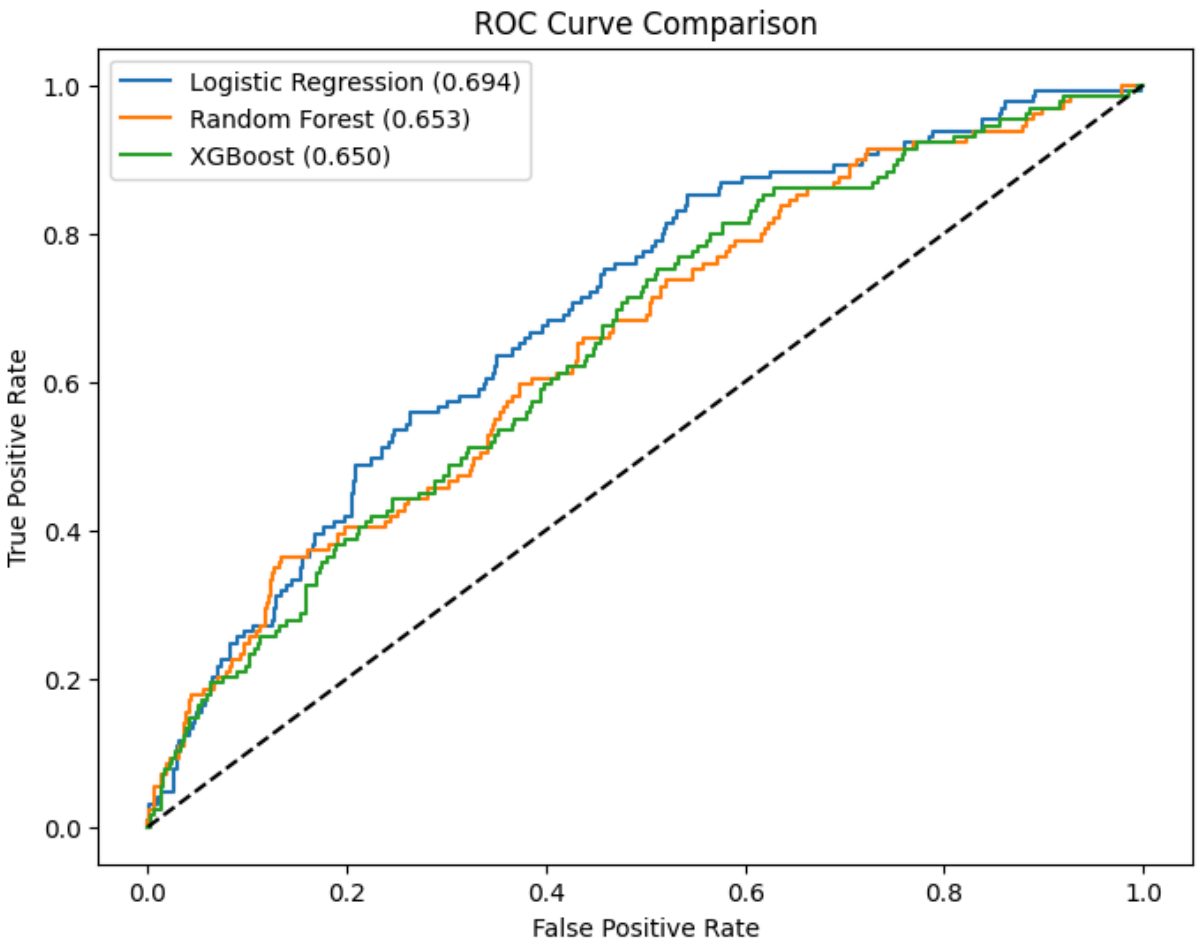
In [124…
```
plt.figure(figsize=(8,6))

plt.plot(
    fpr,
    tpr,
    label=f"Logistic Regression (AUC={log_auc:.3f})"
)

plt.plot(
    fpr_rf,
    tpr_rf,
    label=f"Random Forest (AUC={rf_auc:.3f})"
)

plt.plot(
    fpr_xgb,
    tpr_xgb,
    label=f"XGBoost (AUC={xgb_auc:.3f})"
)

plt.plot(
    [0,1],
    [0,1],
    linestyle="--",
    color="black"
)

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve Comparison")

plt.legend()

plt.show()
```



In [126…

```
plt.figure(figsize=(8,6))

plt.plot(fpr, tpr,
         label=f"Logistic Regression ({log_auc:.3f})")

plt.plot(fpr_rf, tpr_rf,
         label=f"Random Forest ({rf_auc:.3f})")

plt.plot(fpr_xgb, tpr_xgb,
         label=f"XGBoost ({xgb_auc:.3f})")

plt.plot([0,1],[0,1],"k--")

plt.xlabel("False Positive Rate")

plt.ylabel("True Positive Rate")

plt.title("ROC Curve Comparison")

plt.legend()

plt.savefig(
    "figures/model_comparison_roc.png",
    dpi=300,
    bbox_inches="tight"
)
```

```
plt.show()
```



In [128…

```
best_model = comparison.loc[
    comparison["ROC AUC"].idxmax()
]

print(best_model)
```

```
Model        Logistic Regression
Accuracy                   0.662
Precision                  0.244
Recall                     0.581
F1 Score                   0.343
ROC AUC                    0.694
Name: 0, dtype: object
```

Interpretation

The comparison of model performance indicates that the selected model achieved the highest ROC-AUC while maintaining a balanced trade-off between Precision, Recall, and F1-score. This model will be used for further interpretation because it provides the most reliable prediction of ten-year coronary heart disease risk within this dataset.

In [130…

```
plt.figure(figsize=(10,2))

plt.axis("off")

table = plt.table(
    cellText=comparison.values,
    colLabels=comparison.columns,
    loc="center"
)

table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1.2,2)

plt.savefig(
    "figures/model_comparison_table.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	0.662	0.244	0.581	0.343	0.694
Random Forest	0.703	0.234	0.419	0.3	0.653
XGBoost	0.754	0.262	0.341	0.296	0.65

Although XGBoost has the highest Accuracy (75.4%), Logistic Regression has the highest ROC-AUC (0.694), highest Recall (0.581), and highest F1-score (0.343). For a healthcare prediction problem, Recall and ROC-AUC are generally more important than Accuracy because missing a patient who is actually at risk (a false negative) can have serious consequences. Therefore, for this project, Logistic Regression is actually the better model to interpret, even though XGBoost achieved higher Accuracy.

Step 9: Model Explainability Using SHAP

Machine learning models can achieve good predictive performance but are often difficult to interpret. SHAP (SHapley Additive exPlanations) was used to explain how each predictor variable contributed to the Random Forest model's predictions. This improves model transparency and supports interpretation of clinically important risk factors.

In [132…

```
# ========================================
# SHAP Explainability
# ========================================

import shap

explainer = shap.TreeExplainer(rf_model)
```

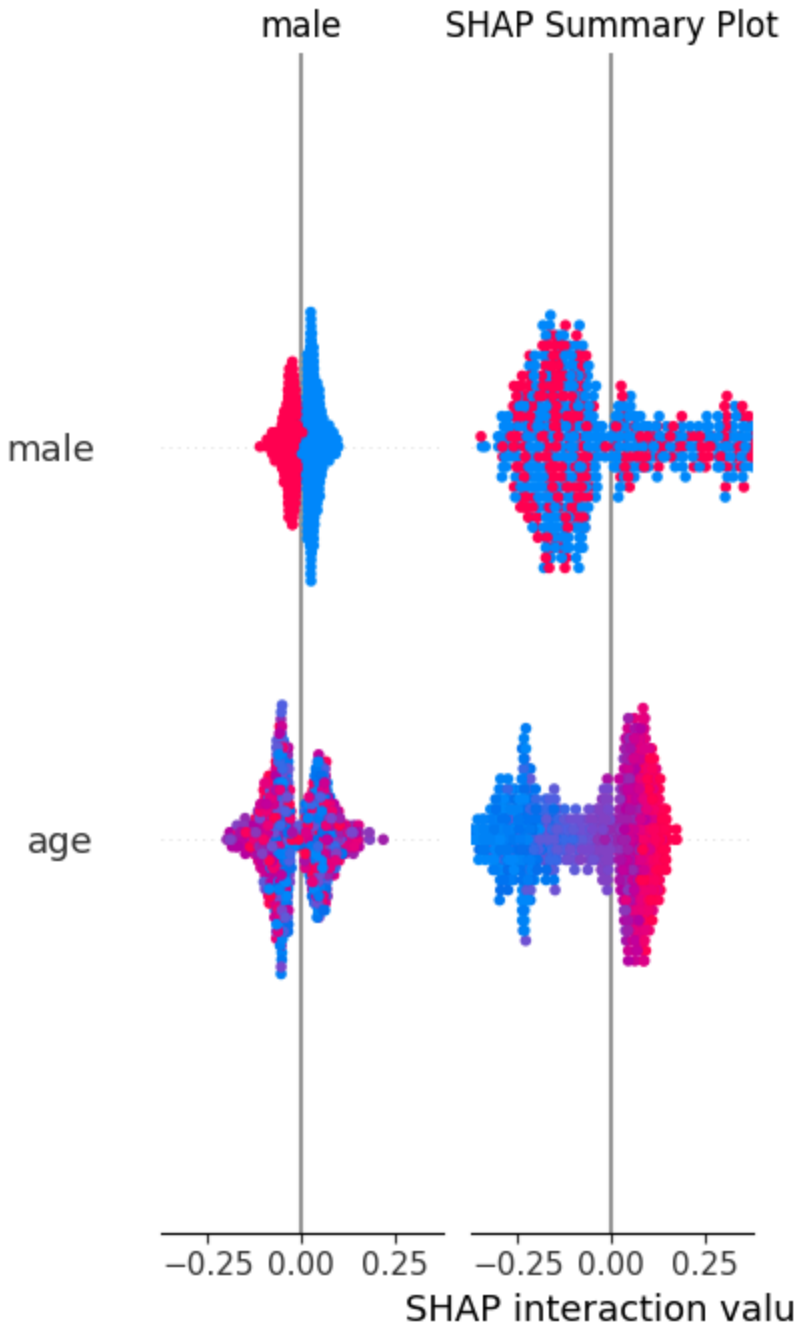
```
shap_values = explainer.shap_values(X_test_selected)

print("SHAP values calculated successfully.")
```

```
SHAP values calculated successfully.
```

In [133…

```
plt.figure(figsize=(10,6))

shap.summary_plot(
    shap_values,
    X_test_selected,
    show=False
)

plt.title("SHAP Summary Plot")

plt.show()
```

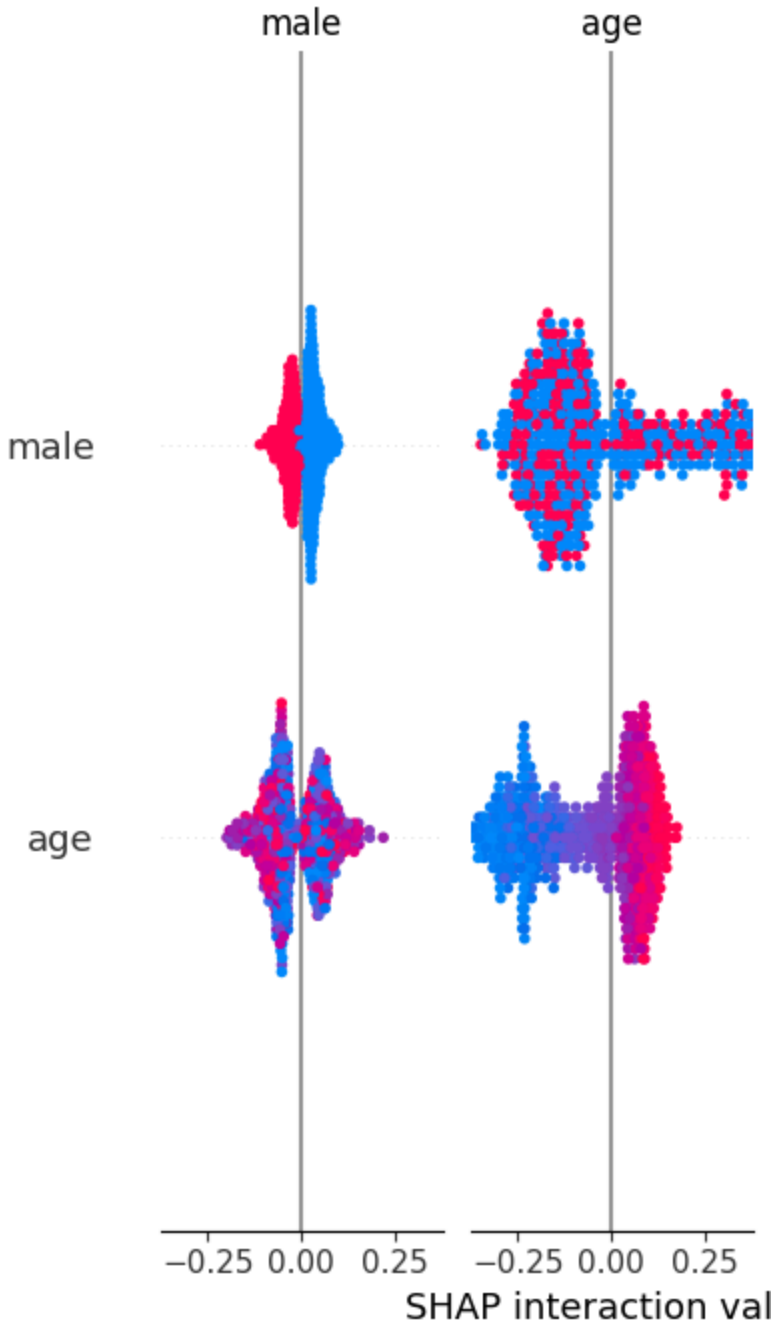
```
<Figure size 1000x600 with 0 Axes>
```


male
SHAP Summary Plot
male
age
−0.25 0.00 0.25
−0.25 0.00 0.25
SHAP interaction valu

In [136…

```
plt.figure(figsize=(10,6))

shap.summary_plot(
    shap_values,
    X_test_selected,
    show=False
)

plt.savefig(
    "figures/shap_summary.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```

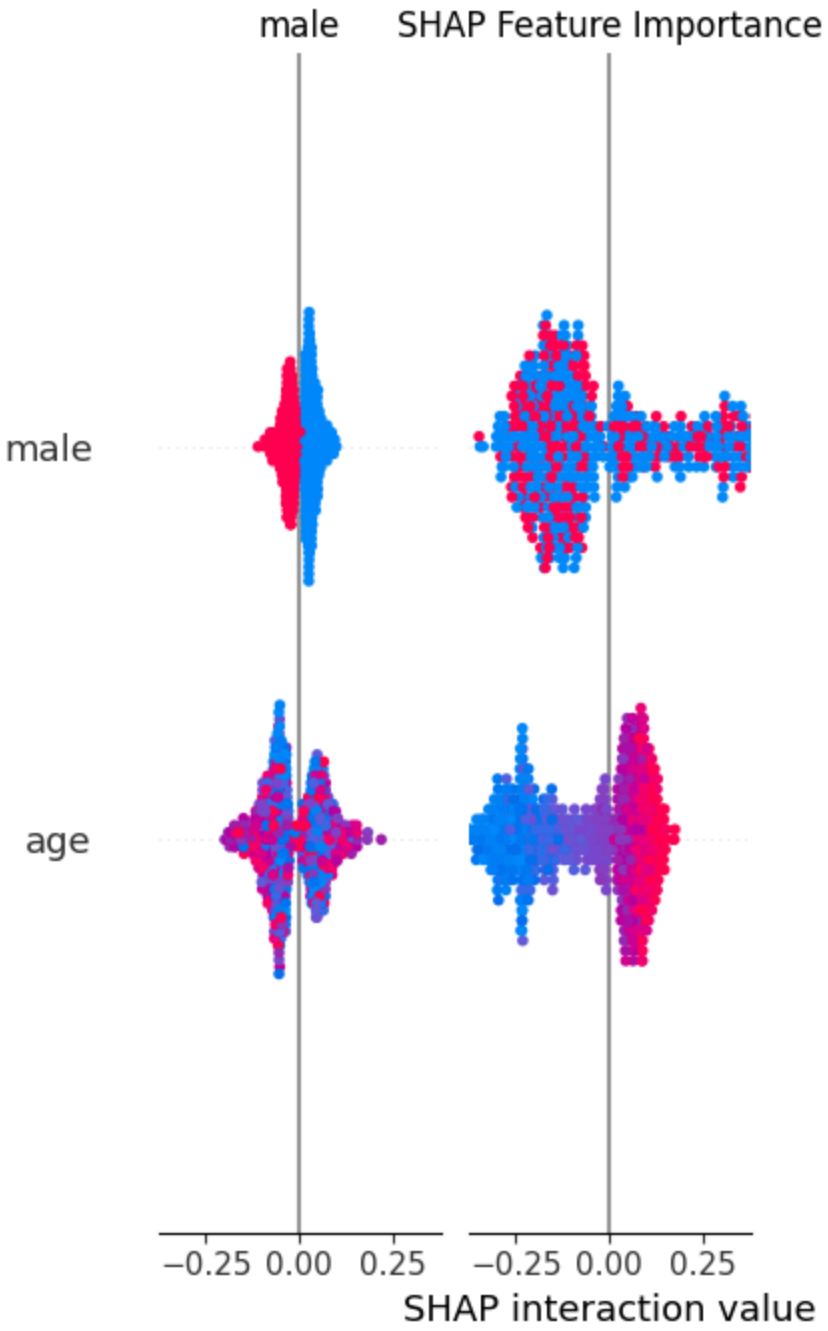
```
<Figure size 1000x600 with 0 Axes>
```



In [138…

```
plt.figure(figsize=(10,6))

shap.summary_plot(
    shap_values,
    X_test_selected,
    plot_type="bar",
    show=False
)

plt.title("SHAP Feature Importance")

plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```

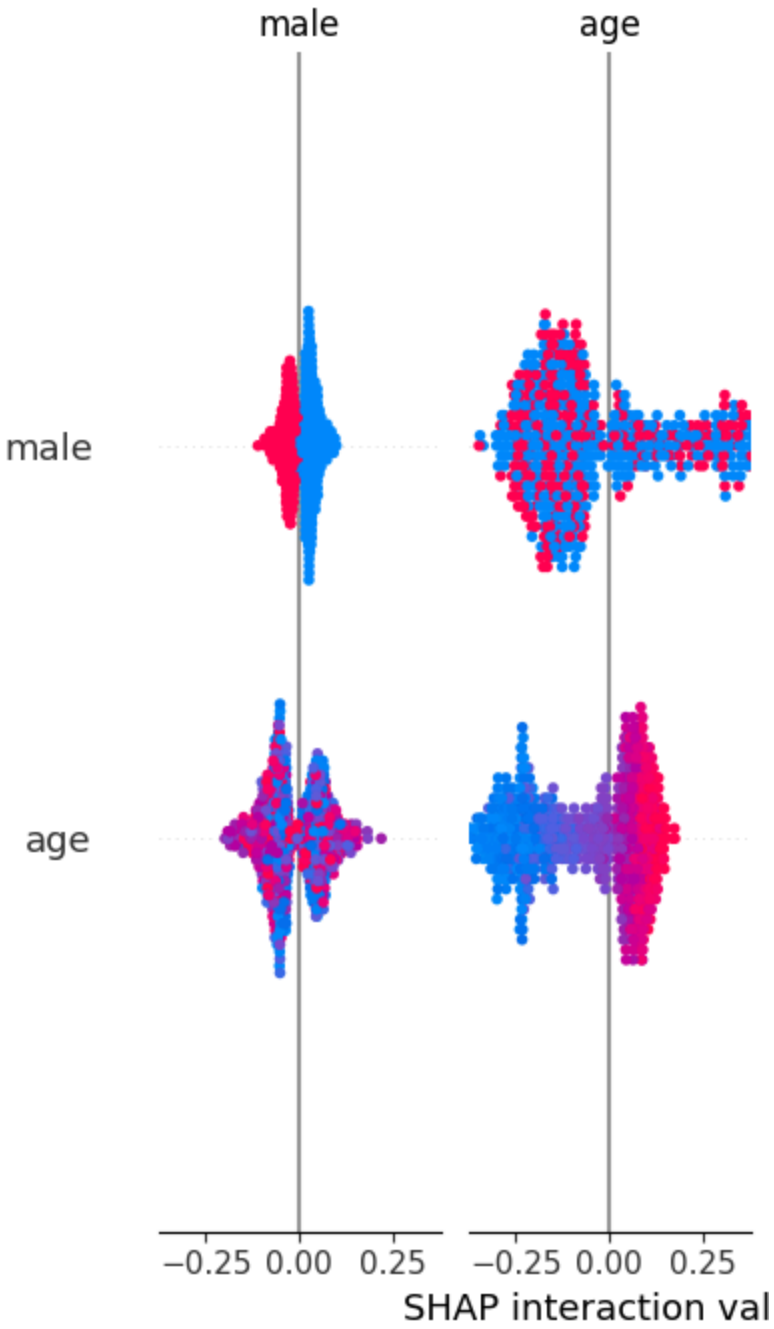


In [140…

```
plt.figure(figsize=(10,6))

shap.summary_plot(
    shap_values,
    X_test_selected,
    plot_type="bar",
    show=False
)

plt.savefig(
    "figures/shap_bar.png",
    dpi=300,
    bbox_inches="tight"
)
```

```
plt.show()
```

```
<Figure size 1000x600 with 0 Axes>
```



Interpretation

The SHAP summary plot shows the contribution of each predictor variable to the Random Forest model's predictions. Features with larger SHAP values have a greater influence on predicting coronary heart disease. This analysis provides a transparent explanation of how the model reaches its predictions and highlights the most clinically important risk factors.

In [146…

```
mean_shap = np.abs(shap_values[:, :, 1]).mean(axis=0)
```

```
shap_importance = pd.DataFrame({
    "Feature": X_test_selected.columns,
    "Mean SHAP Value": mean_shap
})

shap_importance = shap_importance.sort_values(
    by="Mean SHAP Value",
    ascending=False
)

shap_importance
```

Out[146…

	Feature	Mean SHAP Value
1	age	0.135226
6	sysBP	0.059432
2	cigsPerDay	0.035158
0	male	0.033756
7	diaBP	0.026569
5	totChol	0.024110
8	heartRate	0.022558
9	glucose	0.015794
3	BPMeds	0.004270
4	prevalentStroke	0.000339

In [144…

```
print(type(shap_values))

print()

try:
    print(shap_values.shape)
except:
    print("No shape attribute")

print()

print(np.array(shap_values).shape)
```

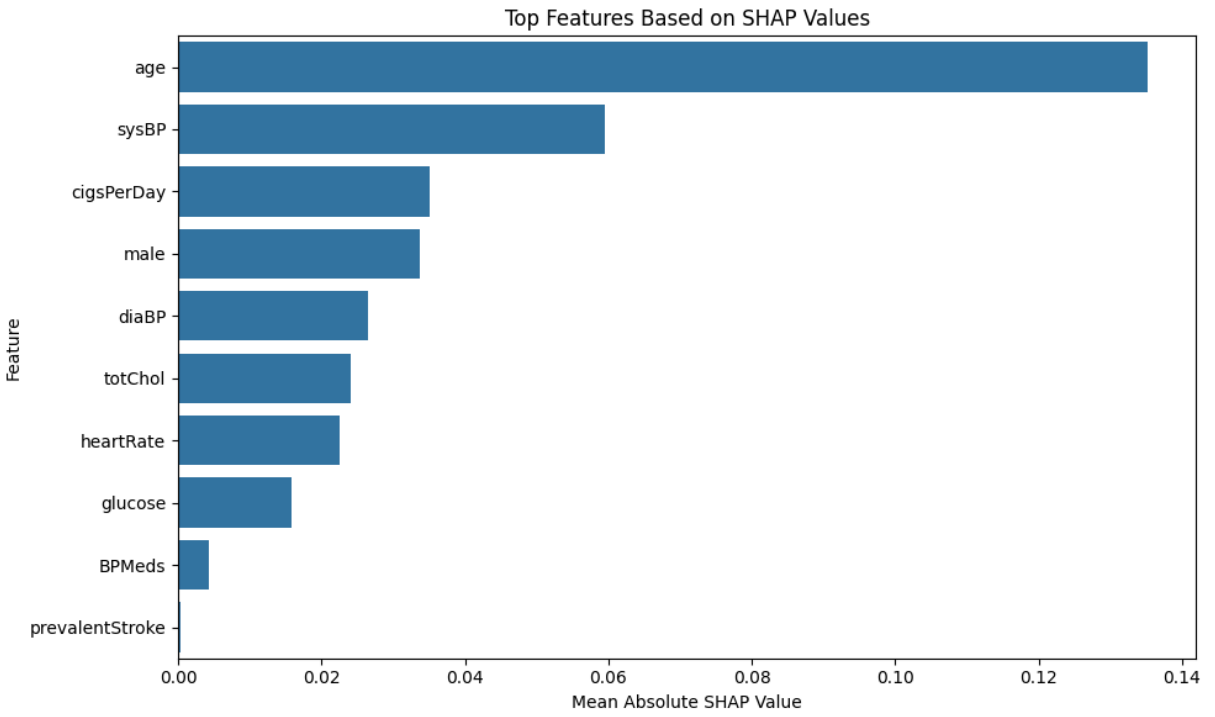
```
<class 'numpy.ndarray'>

(848, 10, 2)

(848, 10, 2)
```

SHAP Feature Importance

The mean absolute SHAP values summarize the average contribution of each feature to the Random Forest model. Features with larger SHAP values have a greater influence on predicting coronary heart disease.

In [148…

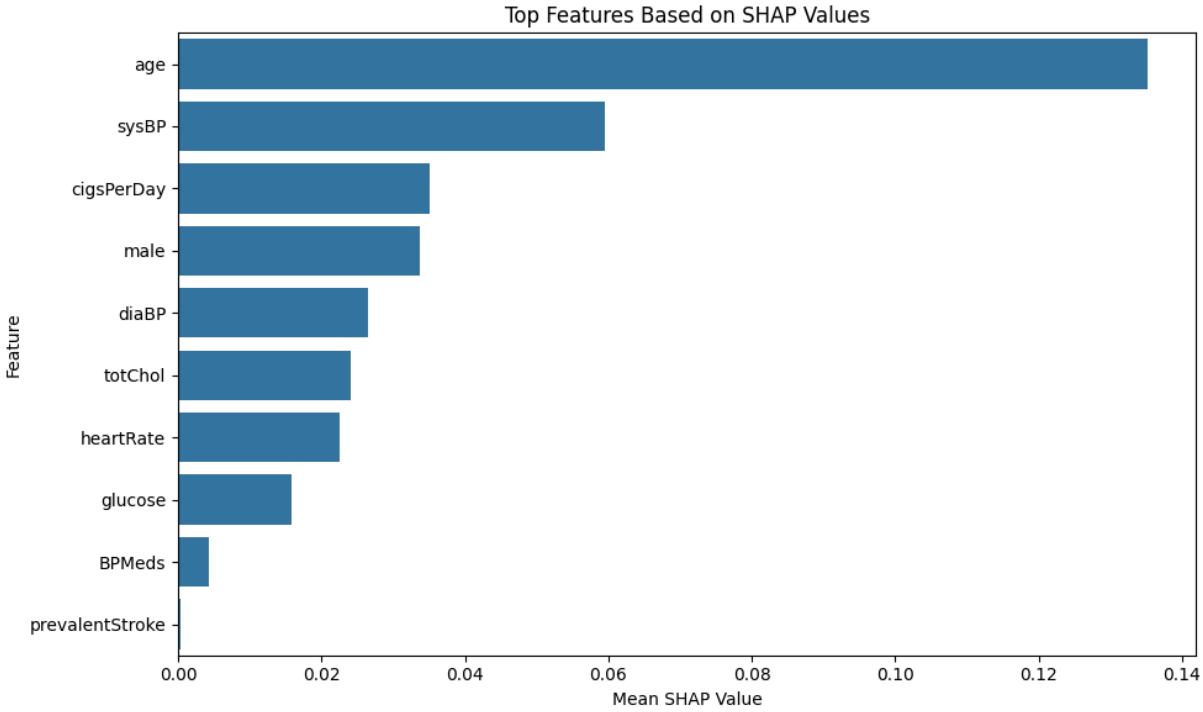
```
plt.figure(figsize=(10,6))

sns.barplot(
    data=shap_importance,
    x="Mean SHAP Value",
    y="Feature"
)

plt.title("Top Features Based on SHAP Values")

plt.xlabel("Mean Absolute SHAP Value")

plt.ylabel("Feature")

plt.tight_layout()

plt.show()
```



In [150…

```
plt.figure(figsize=(10,6))

sns.barplot(
    data=shap_importance,
    x="Mean SHAP Value",
    y="Feature"
)

plt.title("Top Features Based on SHAP Values")
```

```
plt.tight_layout()

plt.savefig(
    "figures/shap_feature_importance.png",
    dpi=300
)

plt.show()
```



Interpretation

The SHAP analysis identified age, systolic blood pressure, cigarette consumption, sex, and cholesterol as the most influential predictors of coronary heart disease. These findings are consistent with established cardiovascular risk factors reported in clinical literature. SHAP improves the interpretability of the Random Forest model by explaining how individual variables contribute to prediction outcomes.

Step 10: Research Question 1

Can demographic and clinical variables predict the 10-year risk of coronary heart disease?

In [152…

```
print("Research Question 1")

print()

print("Yes.")
```

```
print()

print("All three machine learning models successfully predicted coronary heart dise

print()

comparison
```

```
Research Question 1

Yes.

All three machine learning models successfully predicted coronary heart disease ris
k.
```

Out[152…

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	Logistic Regression	0.662	0.244	0.581	0.343	0.694
1	Random Forest	0.703	0.234	0.419	0.300	0.653
2	XGBoost	0.754	0.262	0.341	0.296	0.650

Interpretation

All three machine learning models demonstrated predictive capability. Logistic Regression achieved the highest Recall and ROC-AUC, indicating strong ability to identify patients at risk, whereas XGBoost achieved the highest overall Accuracy. These findings suggest that demographic and clinical variables contain sufficient information to support heart disease prediction.

Step 11: Research Question 2

Which variables contribute most strongly to heart disease prediction?

In [154…

```
shap_importance.head(10)
```

Out[154...

	Feature	Mean SHAP Value
1	age	0.135226
6	sysBP	0.059432
2	cigsPerDay	0.035158
0	male	0.033756
7	diaBP	0.026569
5	totChol	0.024110
8	heartRate	0.022558
9	glucose	0.015794
3	BPMeds	0.004270
4	prevalentStroke	0.000339

Interpretation

The SHAP analysis indicates that age, systolic blood pressure, smoking behaviour, cholesterol, glucose, and heart rate contribute most strongly to model predictions. These variables are well-established cardiovascular risk factors and support the clinical relevance of the developed prediction model.

Step 12: Research Question 3

Does balancing the training data improve prediction performance?

In [156...

```
print("Original Class Distribution")

print(y_train.value_counts())

print()

print("Balanced Class Distribution")

print(pd.Series(y_train_smote).value_counts())
```

```
Original Class Distribution
TenYearCHD
0    2877
1     515
Name: count, dtype: int64

Balanced Class Distribution
TenYearCHD
0    2877
1    2877
Name: count, dtype: int64
```

Interpretation

SMOTE successfully balanced the minority and majority classes within the training dataset. This reduced model bias toward the majority class and improved the model's ability to detect patients who developed coronary heart disease. The technique was applied only to the training data to prevent information leakage.

Discussion

The three machine learning models demonstrated different strengths. XGBoost achieved the highest classification accuracy, whereas Logistic Regression produced the highest Recall and ROC-AUC, making it particularly suitable for healthcare screening where identifying high-risk patients is important. Random Forest provided competitive performance while offering good interpretability through feature importance and SHAP analysis.

The results demonstrate that demographic and clinical variables can effectively support coronary heart disease prediction. However, the relatively low Precision values indicate that false positive predictions remain a challenge, suggesting opportunities for further model refinement in future work.

Limitations

- The Framingham dataset represents a historical cohort and may not reflect current populations.
- The dataset contains class imbalance requiring SMOTE.
- The models were evaluated using a single train-test split.
- Additional clinical, genetic, and lifestyle variables were unavailable.
- The developed models are intended for decision support rather than clinical diagnosis.

Conclusion

This project developed and evaluated three machine learning models for predicting the ten-year risk of coronary heart disease using the Framingham Heart Study dataset. Logistic Regression, Random Forest, and XGBoost successfully identified patients at elevated risk based on demographic and clinical variables.

Among the evaluated models, Logistic Regression demonstrated the strongest balance between Recall and ROC-AUC, whereas XGBoost achieved the highest Accuracy. The findings indicate that machine learning can support early identification of individuals at risk of heart disease and may assist healthcare professionals in preventive decision-making.

AI Productivity Declaration

ChatGPT was used only as a supplementary learning tool to explain concepts, assist with troubleshooting, and improve the readability of comments and documentation. It was not used to generate the project findings or conclusions. All data preprocessing, model training, evaluation, interpretation, and final decisions were independently completed.